VLSI DESIGN VERIFICATION

SystemVerilog Interview Preparation

Module 4: Classes and Object-Oriented Programming

Detailed study notes for Design Verification roles

Concepts from first principles · complete SystemVerilog examples · line-by-line explanations
comparison tables · common mistakes · interview questions with answers · coding exercises

Contents

Module 4 Overview 4

Part 1: Classes, Objects and Handles 5
- 1.1 Introduction to Classes 5
- 1.2 Class Declaration and Object Creation 7
- 1.3 Class Properties and Methods 9
- 1.4 Object Handles and Null Handles 12
- 1.5 Constructors and the new() Method 14
- 1.6 The this Keyword 18

Part 2: Class Members in Depth 21
- 2.1 The Scope Resolution Operator (::) 21
- 2.2 Static Properties 23
- 2.3 Static Methods 26
- 2.4 Extern Methods 29
- 2.5 Constant Properties (const) 31
- 2.6 Local, Protected and Public Access 33
- 2.7 Forward Declaration with typedef class 37
- 2.8 Object Lifetime and Garbage Collection 38

Part 3: Handle Assignment and Object Copying 41
- 3.1 Handle Assignment 41
- 3.2 Shallow Copy 43
- 3.3 Deep Copy 47
- 3.4 Copy Constructors and Copy Methods 49
- 3.5 Comparing Objects 51
- 3.6 The Complete alu_txn Class 51

Part 4: The OOP Pillars, Encapsulation and Inheritance 54
- 4.1 The Four Pillars of OOP 54
- 4.2 Encapsulation 55
- 4.3 Inheritance 57
- 4.4 The super Keyword 61
- 4.5 Method Overriding 65
- 4.6 Method Overloading and Its Limitations 67

Part 5: Polymorphism and Abstraction 71
- 5.1 Upcasting: Base Handles and Derived Objects 71
- 5.2 Virtual Methods and Dynamic Binding 72
- 5.3 Polymorphism 77
- 5.4 Downcasting and $cast 80
- 5.5 Abstract Classes and Pure Virtual Methods 84
- 5.6 Copying in a Class Hierarchy 88
- 5.7 Abstraction 90

Part 6: Module 4 Revision and Practice 92
- 6.1 Revision Summary 92
- 6.2 Interview Questions and Answers 94
- 6.3 Output-Prediction Questions 97
- 6.4 Find-the-Error and Debugging Questions 101

6.5 Coding Exercises with Solutions 105
6.6 Concepts You Must Understand Before Moving Forward 111
6.7 What Comes Next 112

Module 4 Overview

Module 4 teaches classes and object-oriented programming (OOP) in SystemVerilog from the very beginning. Every modern testbench, and the whole of UVM, is built from classes, so this module is the foundation for Modules 5 to 11.

What this module covers

Part	Topics
Part 1	What a class is, objects, handles, `null`, properties and methods, constructors, `new()`, `this`
Part 2	The scope resolution operator `::`, static properties and methods, `extern` methods, `const` properties, `local` / `protected` / public access, forward `typedef class`, object lifetime and garbage collection
Part 3	Handle assignment, shallow copy, deep copy, copy methods ("copy constructors"), comparing objects, the complete `alu_txn` class
Part 4	The four OOP pillars, encapsulation, inheritance (single and multilevel), `super`, method overriding, why method overloading does not exist
Part 5	Upcasting, virtual methods and dynamic binding, polymorphism, downcasting with `$cast`, abstract (virtual) classes, pure virtual methods, abstraction
Part 6	Revision summary, interview questions, output-prediction, find-the-error, debugging, coding exercises with solutions

What you need from earlier modules

From	You should already know
Module 1	`bit`, `logic`, `int`, `string`, `enum`, `struct`, `typedef`, the difference between static and dynamic casting, `===` versus `==`
Module 2	Fixed arrays, dynamic arrays, queues (`push_back`, `foreach`)
Module 3	Functions and tasks, arguments and default values, `automatic` versus `static` lifetime, `void` functions

Conventions. The naming rules of Module 1 continue: types end in `_t`, enums in `_e`, small demonstration modules are called `tb_<topic>`. The running example is still the 8-bit ALU; in this module we build its transaction class, `alu_txn`, step by step. Classes are simulation-only: nothing in this module is synthesizable.

> **Note:** No simulator was available when these notes were written. Every "Expected output" block was derived by hand from the IEEE 1800-2017 rules. Run each example yourself (the files are in `examples/module-04/`) and investigate any difference.

Part 1: Classes, Objects and Handles

1.1 Introduction to Classes

In plain English

In Module 1 a `struct` let us group related data under one name: an address, some data and a read/write flag became one "request". But a struct is only data. The code that prints a request, copies it, compares it or checks it lives somewhere else, in loose functions that must be handed the struct every time.

A **class** groups the data *and* the code that works on that data into one unit. A class called `packet` contains the fields of a packet and also the functions `print()`, `copy()` and `compare()` that know how to handle a packet.

A class by itself is only a description, like the architectural drawing of a house. Nobody can live in a drawing. From one drawing you can build many houses, each with its own address and its own furniture. Each house is an **object**. To find a particular house you need its address written on a piece of paper; that piece of paper is a **handle**.

Everyday idea	OOP term	In SystemVerilog
Drawing of a house	Class	`class packet; ... endclass`
A house that was built	Object (instance)	Created by `new()`
Paper with the address of one house	Handle	A variable of the class type: `packet p;`
Blank paper, no address on it	Null handle	`p == null`
Rooms and furniture	Properties (data members)	Variables declared inside the class
Things you can do in the house	Methods	Functions and tasks declared inside the class

Technical definition

- A **class** is a user-defined data type that encapsulates **properties** (variables) and **methods** (functions and tasks that operate on those variables).
- An **object** is an instance of a class. It is created dynamically, at run time, by calling the constructor `new()`. Memory for it is allocated at that moment.
- A **handle** is a variable whose type is a class. It holds a reference to an object, or the special value `null` when it refers to no object. A handle is like a safe pointer: you cannot do arithmetic on it and it can only refer to objects of a compatible class.

Why classes are needed in verification

Need in a testbench	Why modules and structs are not enough	What a class provides
Thousands of stimulus items (transactions) created during the run	Module instances are fixed at elaboration; they cannot be created at run time	Objects are created with `new()` whenever needed and freed automatically

Need in a testbench	Why modules and structs are not enough	What a class provides
Data and its operations kept together	A struct has no methods	Properties and methods in one unit
Random stimulus with rules	A struct has no `randomize()` method and no constraint blocks	`rand` properties, `constraint` blocks, `randomize()` (Module 5)
Reuse: a new test that is "the old one, slightly changed"	Modules cannot be extended; you copy and edit	Inheritance: `class new_test extends old_test`
One driver that handles many kinds of transaction	Requires `case` statements everywhere	Polymorphism through virtual methods
Passing data between components cheaply	A struct is copied on every assignment	Only the handle is passed; the object is not copied

Class versus module versus struct

Property	`module`	`struct`	`class`
Purpose	Hardware hierarchy, testbench top	Group data	Group data and behaviour
Created	At elaboration, before time 0; fixed for the whole run	When the variable is declared	At run time with `new()`
Destroyed	Never	With its variable	Automatically, when no handle refers to it
Contains methods	Tasks and functions, yes	No	Yes
Contains `always`, `assign`, `initial`	Yes	No	No
A variable of this type holds	n/a (modules are instantiated, not stored)	The data itself	A handle (reference)
Assignment `a = b`	n/a	Copies all members	Copies the handle only
Inheritance and polymorphism	No	No	Yes
Randomization with constraints	No	No	Yes
Synthesizable	Yes	Yes	No
Typical use	DUT, `tb_top`, interfaces' containers	Signal bundles, bit fields	Transactions and every testbench component

Where classes may be declared

A class can be declared inside a `package` (the normal place in a real project, see Module 11), inside a `module`, `program` or `interface`, inside another class, or at file level outside everything (the "compilation unit" scope). To keep each example in one self-contained file, the examples in this module declare classes inside the demonstration module.

Interview questions

Q1. What is a class? A user-defined data type that encapsulates properties (data) and methods (tasks and functions) that operate on that data. It is a template from which objects are created at run time.

Q2. What is the difference between a class and an object? A class is the type definition; it occupies no memory for data. An object is an instance of the class, created by `new()`, with its own copy of every non-static property.

Q3. What is the difference between a class and a struct? A struct holds only data, is allocated statically, and a struct variable contains the data itself, so assignment copies everything. A class has data and methods, is created dynamically with `new()`, is accessed through a handle, and supports randomization, inheritance and polymorphism. Structs are synthesizable; classes are not.

Q4. What is the difference between a class and a module? A module is a static hardware-style container that exists for the entire simulation and can hold `always` blocks, continuous assignments and instances. A class is a dynamic software-style type whose objects are created and destroyed during simulation. A class cannot contain `always` or `assign`, and a module cannot be extended or created at run time.

Q5. Are classes synthesizable? No. Classes are a verification construct.

1.2 Class Declaration and Object Creation

Syntax

```
class class_name;
  // properties
  data_type property_name;

  // methods
  function return_type method_name(arguments);
    ...
  endfunction

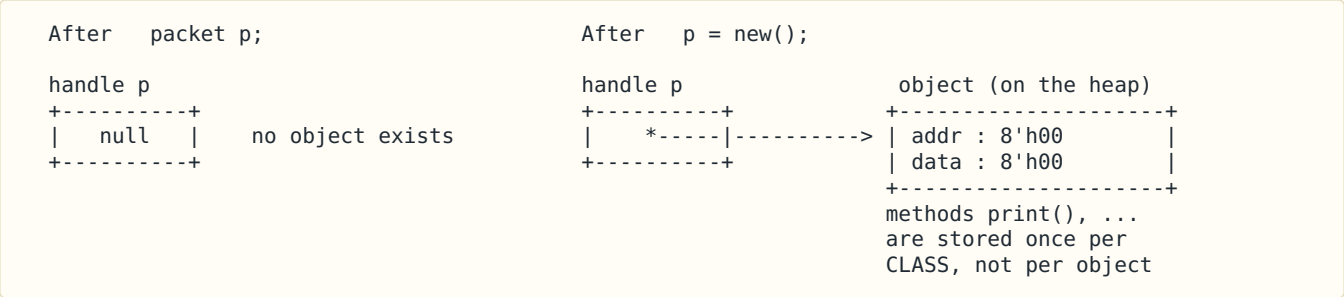
  task task_name(arguments);
    ...
  endtask
endclass
```

Creating and using an object always takes three steps:

```
packet p;        // 1. declare a handle        (p is null)
p = new();       // 2. construct an object     (memory is allocated, p refers to it)
p.addr = 8'h10;  // 3. use the object through the handle
```

Steps 1 and 2 may be combined: `packet p = new();`. The parentheses after `new` are optional when no arguments are passed, so `p = new;` is the same as `p = new();`.

How it works internally



What `new()` does, in order:

1. Allocates memory for one object of the class.
2. Gives every property its initial value: the default of its type (Module 1, Section 4.2), or the initialiser written in its declaration.
3. Executes the body of the constructor function `new`, if the class defines one.
4. Returns a reference to the object, which is stored in the handle on the left of the assignment.

Methods are code, and code is not duplicated. One thousand `packet` objects contain one thousand copies of `addr` and `data` but share one copy of `print()`.

Complete example

tb_class_basic.sv

```
module tb_class_basic;

  class packet;
    bit [7:0] addr;
    bit [7:0] data;

    function void print(string tag = "");
      $display("[%s] addr=%h data=%h", tag, addr, data);
    endfunction
  endclass

  packet p1;                        // a handle; no object yet

  initial begin
    if (p1 == null) $display("p1 is null before new()");

    p1 = new();                     // the object is created here
    p1.print("after new");

    p1.addr = 8'h10;
    p1.data = 8'hAB;
    p1.print("after set");
  end

endmodule
```

EXPECTED OUTPUT

```
p1 is null before new()
[after new] addr=00 data=00
[after set] addr=10 data=ab
```

Line by line

Line	Explanation
`class packet;` ... `endclass`	Declares a new data type called `packet`. No memory is allocated for `addr` or `data` yet.
`bit [7:0] addr;` `bit [7:0] data;`	Properties. Each object will have its own copy.
`function void print(string tag = "");`	A method. Inside it, `addr` and `data` mean "the `addr` and `data` of the object this method was called on".
`packet p1;`	Declares a handle of type `packet`. Its value is `null`.
`if (p1 == null)`	A handle can be compared with `null`. This is how you test whether an object exists.
`p1 = new();`	Allocates a `packet` object and stores its reference in `p1`.
`p1.print("after new");`	Calls the method on the object. The properties are 0 because `bit` defaults to 0.
`p1.addr = 8'h10;`	Writes a property through the handle, using the dot operator.

> **Common mistake:** Forgetting step 2. Declaring `packet p1;` does **not** create an object. Using `p1.addr` before `p1 = new();` is a run-time fatal error: "null object access" (also reported as "null pointer dereference"). It compiles without any warning. This is the most common run-time error in class-based testbenches.

> **Common mistake:** Writing a declaration after a statement inside a `begin` ... `end` block. All declarations (`packet p;`) must come before the first statement of the block.

Interview questions

Q1. How do you create an object? Declare a handle of the class type and assign it the result of the constructor: `packet p; p = new();` or in one line `packet p = new();`.

Q2. What does `new()` do? It allocates memory for an object, initialises the properties (type default or declaration initialiser), runs the constructor body, and returns the object's reference.

Q3. What happens if you use a handle without calling `new()`? The handle is `null`. Accessing a non-static property or calling a non-static method through it causes a fatal run-time error (null object access). The compiler does not detect it.

Q4. When is the memory for a class allocated? When `new()` is called, at run time, not when the class or the handle is declared.

1.3 Class Properties and Methods

Properties

A **property** is a variable declared inside a class. It can be of any data type: integral types, `real`, `string`, enums, structs, arrays of every kind, events, virtual interfaces, and handles of other classes (or of the same class, which is how linked lists are built).

```
class example;
  bit [7:0]  addr;                // integral
  int        delay = 5;           // with an initial value
  string     name  = "item";
  bit [7:0]  payload [];          // dynamic array (Module 2)
  int        history [$];         // queue
  example    next;                // handle of the same class
endclass
```

A property may have an initialiser in its declaration. The initialiser is applied every time an object is created, before the constructor body runs.

Methods

A **method** is a function or a task declared inside a class.

	Function in a class	Task in a class
Can consume simulation time (`#`, `@`, `wait`)	No	Yes
Returns a value	Yes (or `void`)	No (uses `output` / `ref` arguments)
Typical use	`print`, `copy`, `compare`, `get_*`, calculations, `new`	`run`, `drive`, `wait_for_reset`: anything that follows the clock

Two rules are specific to class methods:

1. **Class methods always have automatic lifetime.** Each call has its own copies of the arguments and local variables, so methods are re-entrant and can be called from several threads at once. Declaring a class method with static lifetime (`function static ...`) is illegal. (In a module, tasks and functions are static by default; see Module 3.)
2. **A method can use every property and every other method of its own class directly**, without a handle. The compiler knows they belong to the object on which the method was called.

Accessing class members

Members are accessed with the dot operator through a handle:

```
handle.property            // read or write a property
handle.method(args)        // call a method
handle.sub_handle.property // chain through a nested object
```

Complete example

tb_class_methods.sv

```
module tb_class_methods;

  class counter;
    int    count;
    string name = "cnt";

    function void incr(int step = 1);
      count += step;
    endfunction

    function int get();
      return count;
    endfunction

    task wait_then_incr(int delay);
      #delay;
      count++;
      $display("[%0t] %s: count=%0d", $time, name, count);
    endtask
  endclass

  counter c;

  initial begin
    c = new();
    c.incr();                    // step defaults to 1
    c.incr(5);
    $display("[%0t] %s: count=%0d", $time, c.name, c.get());

    c.wait_then_incr(10);        // a task: simulation time advances
    c.wait_then_incr(5);
  end

endmodule
```

EXPECTED OUTPUT

```
[0] cnt: count=6
[10] cnt: count=7
[15] cnt: count=8
```

Line by line

Line	Explanation
`int count;`	Starts at 0 in every new object.
`string name = "cnt";`	A declaration initialiser: every new object starts with this name.
`function void incr(int step = 1);`	A method with a default argument. It changes the object's own `count`.
`function int get();`	Returns a value. Called a "getter".
`task wait_then_incr(int delay);`	A task may contain `#delay`. A function may not.
`c.incr(); c.incr(5);`	0 + 1 + 5 = 6.
`c.wait_then_incr(10);`	The calling `initial` block is suspended for 10 time units inside the task.

> **Verification application:** In the layered testbench of Module 9, every component is a class with a `task run()`. The driver's `run()` contains a `forever` loop that waits for a clock edge and drives pins; the transaction class has only functions (`print`, `copy`, `compare`) because a transaction is passive data.

Interview questions

Q1. Can a class contain tasks as well as functions? Yes. Functions are used for operations that take no simulation time; tasks for operations that wait on clocks, delays or events.

Q2. What is the default lifetime of a class method? Automatic. It cannot be changed to static.

Q3. Can a class contain an `always` block or an `initial` block? No. A class contains only declarations: properties, methods, constraints, covergroups, typedefs and parameters. Activity is started by calling a task of the object from an `initial` block (or from `fork`), typically a `run()` task with a `forever` loop.

Q4. Can a property of a class be a handle to the same class? Yes. `class node; node next; endclass` is legal and is used to build linked data structures.

1.4 Object Handles and Null Handles

In plain English

A handle is not the object. It is the address of the object. Several handles can hold the same address, and a handle can be changed to hold a different address. Understanding this one fact prevents most OOP bugs in testbenches.

Technical definition

- A **handle** is a variable of a class type that stores a reference to an object of that class (or of a class derived from it), or `null`.
- `null` is the default value of every handle. It means "refers to no object".
- Operations allowed on handles: assignment (`h1 = h2`, `h = null`), equality comparison (`==`, `!=`, `===`, `!==`, all of which compare the *references*), use as a condition after comparison with `null`, and member access with the dot operator.
- Operations not allowed: arithmetic, casting to or from integers, taking the address of a variable.

Handle versus C pointer

Property	C pointer	SystemVerilog handle
Arithmetic (`p + 1`)	Allowed	Not allowed
Can point to any memory	Yes	Only to an object of a compatible class
Cast from an integer	Allowed	Not allowed
Dangling reference (pointing to freed memory)	Possible	Impossible: an object lives as long as any handle refers to it
Memory must be freed by the programmer	Yes (`free`)	No, garbage collection is automatic
Null access	Undefined behaviour	Clean run-time fatal error

Complete example

tb_handles.sv

```
module tb_handles;

  class packet;
    bit [7:0] addr;
  endclass

  packet p1, p2, p3;

  initial begin
    p1 = new();  p1.addr = 8'h01;        // object A
    p2 = new();  p2.addr = 8'h02;        // object B

    p3 = p1;                             // p3 refers to object A too
    p3.addr = 8'h33;
    $display("1. p1.addr=%h p2.addr=%h p3.addr=%h", p1.addr, p2.addr, p3.addr);

    if (p1 == p3) $display("2. p1 and p3 refer to the same object");
    if (p1 != p2) $display("3. p1 and p2 refer to different objects");

    p1 = null;                           // object A is still reachable through p3
    $display("4. p3.addr=%h (object A is alive)", p3.addr);

    p3 = p2;                             // nothing refers to object A now
    $display("5. p3.addr=%h (p3 now refers to object B)", p3.addr);

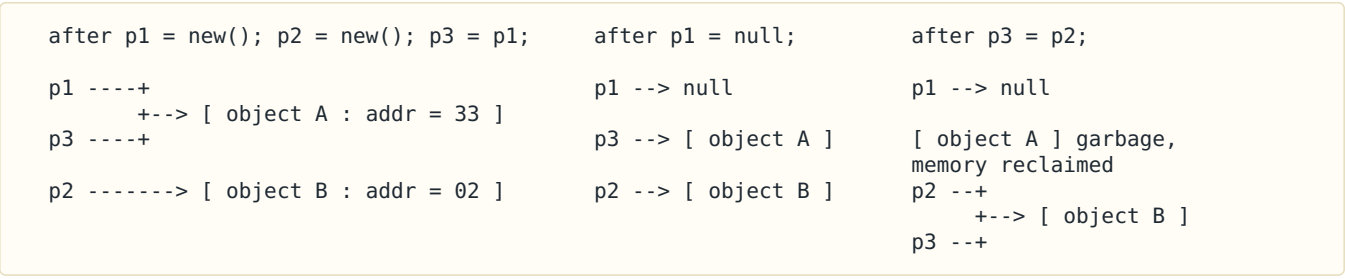
    if (p1 == null) $display("6. p1 is null: p1.addr would be a fatal error");
    // p1.addr = 8'h55;                  // run-time error: null object access
  end

endmodule
```

EXPECTED OUTPUT

```
1. p1.addr=33 p2.addr=02 p3.addr=33
2. p1 and p3 refer to the same object
3. p1 and p2 refer to different objects
4. p3.addr=33 (object A is alive)
5. p3.addr=02 (p3 now refers to object B)
6. p1 is null: p1.addr would be a fatal error
```

Execution, step by step



Line	Explanation
`p3 = p1;`	Copies the reference, not the object. There are still only two objects.
`p3.addr = 8'h33;`	Changes object A. `p1.addr` shows 33 because `p1` refers to the same object.
`p1 == p3`	Handle comparison: true when both refer to the same object. It does not compare contents.
`p1 = null;`	`p1` lets go of object A. The object survives because `p3` still refers to it.

Line	Explanation
`p3 = p2;`	Now no handle refers to object A. It can never be reached again, so the simulator reclaims its memory (Section 2.8).
Commented line	Accessing a property through a null handle stops the simulation with a fatal error.

Protecting against null handles

```
if (txn == null) $fatal(1, "driver received a null transaction");

if (txn != null && txn.addr == 8'h10) ...   // txn.addr is read only if txn exists
```

Interview trap: "What does `p1 == p2` compare when `p1` and `p2` are class handles?" It compares the handles: the result is 1 only if both refer to the *same* object (or both are `null`). Two different objects with identical contents are not equal under `==`. To compare contents you write a `compare()` method (Section 3.5).

Common mistake: Assuming that a handle that is a class property is constructed automatically. A property `header hdr;` inside a class is `null` after the outer object is created, unless the constructor (or a declaration initialiser) calls `hdr = new();`.

Interview questions

Q1. What is a handle? A variable of a class type that holds a reference to an object, or `null`.

Q2. What is the default value of a handle? `null`.

Q3. What is the difference between a handle and an object? The object is the block of memory holding the property values, created by `new()`. The handle is a named variable that refers to it. One object can have many handles; one handle refers to at most one object at a time.

Q4. How is a handle different from a C pointer? A handle is type-safe: no arithmetic, no casting from integers, it can refer only to objects of a compatible class, it can never dangle, and the memory it refers to is reclaimed automatically.

Q5. What happens after `h2 = h1;`? Both handles refer to the same object. No new object is created. A change made through one handle is visible through the other.

Q6. How do you check that a handle is valid before using it? Compare it with `null`: `if (h != null) ...`.

1.5 Constructors and the new() Method

In plain English

When a house is built, somebody must do the initial setup: fit the doors, connect the electricity. The **constructor** is the function that does the initial setup of an object. It runs automatically, once, at the moment the object is created.

Technical definition

- The constructor is a special function named `new`. It is called when an object is created with `handle = new(arguments);`.
- It has **no return type** (not even `void`). It implicitly returns the reference to the newly created object.
- It may take arguments, including arguments with default values.
- A class can have **only one** constructor, because SystemVerilog has no method overloading (Section 4.6).
- It is a function, so it cannot contain timing controls.
- It cannot be declared `static` or `virtual`.

Default constructor and custom constructor

	Default (implicit) constructor	Custom (user-defined) constructor
Who writes it	The compiler provides it when the class does not define `new`	You write `function new(...); ... endfunction`
Arguments	None	Any, with or without defaults
What it does	Allocates the object; sets each property to its type default or declaration initialiser; in a derived class also calls the base-class constructor	The same, and then runs your statements
Typical use	Simple data classes	Setting initial values, creating nested objects, receiving a name, a virtual interface or mailbox handles

Syntax

```
class packet;
  bit [7:0] addr;
  bit [7:0] data;

  function new(bit [7:0] a = 8'h00, bit [7:0] d = 8'hFF);   // custom constructor
    addr = a;
    data = d;
  endfunction
endclass

packet p1 = new();              // both defaults
packet p2 = new(8'h10);         // a = 10, d = default
packet p3 = new(8'h20, 8'h55);  // positional arguments
packet p4 = new(.d(8'h77));     // argument passed by name
```

Order of events inside new()

```
p = new(8'h10, 8'h55);

1. memory for the object is allocated
2. (derived classes only) the base-class constructor runs first      -> Section 4.4
3. every property gets its initial value:
       - the initialiser from its declaration, if there is one
       - otherwise the default of its data type (0, X, "", null ...)
4. the statements in the body of function new() execute
5. the reference to the object is returned and stored in p
```

Because step 3 comes before step 4, the constructor body can read the declaration initialisers and may overwrite them.

Complete example

tb_constructor.sv

```
module tb_constructor;

  class packet;
    bit [7:0] addr;
    bit [7:0] data;
    int       len = 4;                    // declaration initialiser

    function new(bit [7:0] a = 8'h00, bit [7:0] d = 8'hFF);
      $display("   new() starts with len=%0d", len);
      addr = a;
      data = d;
      len  = len * 2;
    endfunction

    function void print(string tag = "");
      $display("[%s] addr=%h data=%h len=%0d", tag, addr, data, len);
    endfunction
  endclass

  packet p1, p2, p3, p4;

  initial begin
    p1 = new();              p1.print("p1");
    p2 = new(8'h10);         p2.print("p2");
    p3 = new(8'h20, 8'h55);  p3.print("p3");
    p4 = new(.d(8'h77));     p4.print("p4");
  end

endmodule
```

EXPECTED OUTPUT

```
   new() starts with len=4
[p1] addr=00 data=ff len=8
   new() starts with len=4
[p2] addr=10 data=ff len=8
   new() starts with len=4
[p3] addr=20 data=55 len=8
   new() starts with len=4
[p4] addr=00 data=77 len=8
```

Line by line

Line	Explanation
`int len = 4;`	Applied to every new object before the constructor body runs.
`function new(bit [7:0] a = 8'h00, bit [7:0] d = 8'hFF);`	No return type is written. Both arguments are optional because they have defaults.
`$display(" new() starts with len=%0d", len);`	Prints 4, proving that the declaration initialiser has already been applied.
`len = len * 2;`	The constructor may modify the initial value: 8.
`p1 = new();`	Uses both defaults.
`p2 = new(8'h10);`	First argument given, second defaulted.
`p4 = new(.d(8'h77));`	Named argument binding skips `a`, which takes its default.

Constructors that build nested objects and receive connections

In a real testbench the constructor has two typical jobs: create the objects this object owns, and store the connections it was given.

```
class alu_driver;
  virtual alu_if      vif;            // connection to the DUT pins        (Module 8)
  mailbox #(alu_txn)  gen2drv_mbx;    // connection to the generator       (Module 7)

  function new(virtual alu_if vif, mailbox #(alu_txn) gen2drv_mbx);
    this.vif         = vif;           // "this" is explained in the next section
    this.gen2drv_mbx = gen2drv_mbx;
  endfunction
endclass
```

Interview trap: "What is the return type of `new`?" It has none; writing `function void new()` is a syntax error. The constructor implicitly returns the handle of the created object.

Interview trap: "Can a class have several constructors with different argument lists?" No. SystemVerilog does not support overloading. One `new` with default arguments covers the common cases.

Common mistake: Declaring a constructor with required arguments and then calling `new()` without them. This is a compile error. Give the arguments default values if the object should also be constructible without them.

Common mistake: Putting a delay or `@(posedge clk)` in `new`. The constructor is a function and cannot consume time. Put time-consuming setup in a task such as `run()` or `reset()`.

new() versus new[]

	`new()`	`new[]`
Purpose	Constructs a class object	Allocates a dynamic array (Module 2)
Brackets	Parentheses, arguments of the constructor	Square brackets, the number of elements
Example	`p = new(8'h10);`	`arr = new[20];`

A dynamic array of handles needs both: `arr = new[4];` creates four null handles, and then `foreach (arr[i]) arr[i] = new();` creates the four objects.

Interview questions

Q1. What is a constructor? A special function named `new` that is called automatically when an object is created. It allocates the object and initialises it.

Q2. What is the default constructor? The implicit constructor supplied by the compiler when the class does not define `new`. It takes no arguments and sets every property to its declaration initialiser or type default.

Q3. Can a constructor have arguments? Can it return a value? It can have arguments, with or without default values. It has no declared return type and cannot return a user value; it implicitly returns the object handle.

Q4. Can the constructor be a task? No. It is always a function, so it cannot contain delays or event controls.

Q5. In which order are declaration initialisers and the constructor body executed? Declaration initialisers first, then the constructor body. In a derived class the base-class constructor runs before both (Section 4.4).

Q6. What is the difference between `new()` and `new[]`? `new()` constructs a class object. `new[n]` sets the size of a dynamic array.

Q7. Can `new` be virtual or static? No.

1.6 The this Keyword

In plain English

Inside a method, `this` means "the object I was called on". Most of the time you do not need to write it, because a plain property name already means "my own property". You need it in two situations: when a local name hides a property name, and when the object must hand its own handle to somebody else.

Technical definition

`this` is a predefined handle, available inside every non-static method of a class, that refers to the object on which the method was invoked. `this.x` always means the class property `x` of the current object, whatever local variables or arguments are in scope.

Why it is needed: name shadowing

SystemVerilog resolves a name by searching from the innermost scope outward: first the local variables and arguments of the method, then the class properties. If an argument has the same name as a property, the plain name refers to the **argument**, and the property is hidden ("shadowed").

```
function new(bit [7:0] addr);
  addr = addr;          // argument = argument    -> the property is never written
  this.addr = addr;     // property = argument    -> correct
endfunction

name lookup for the plain name "addr":
   1. method scope (arguments, local variables)   <- found here: the argument
   2. class scope (properties)                     <- never reached
```

Using the same name for the argument and the property is the normal coding style, because it avoids inventing names such as `a`, `addr_i` or `new_addr`. That style depends on `this`.

Uses of this

Use	Example
Distinguish a property from an argument or local variable of the same name	`this.addr = addr;`
Pass the current object to another object or function	`scoreboard.register(this);`
Return the current object, for example to allow call chaining	`return this;`
Make it explicit that a name is a class member (readability)	`this.count++;`

`this` cannot be used in a static method, because a static method is not called on any object (Section 2.3).

Complete example

tb_this.sv

```
module tb_this;

  class packet;
    bit [7:0] addr;
    bit [7:0] data;

    function new(bit [7:0] addr, bit [7:0] data);
      this.addr = addr;          // property = argument
      this.data = data;
    endfunction

    function void bad_set(bit [7:0] addr);
      addr = addr;               // BUG: assigns the argument to itself
    endfunction

    function void good_set(bit [7:0] addr);
      this.addr = addr;
    endfunction

    function packet get_self();
      return this;               // returns a handle to the current object
    endfunction
  endclass

  packet p, q;

  initial begin
    p = new(8'h11, 8'h22);
    $display("1. after new      : addr=%h data=%h", p.addr, p.data);

    p.bad_set(8'h99);
    $display("2. after bad_set  : addr=%h", p.addr);

    p.good_set(8'h99);
    $display("3. after good_set : addr=%h", p.addr);

    q = p.get_self();
    if (q == p) $display("4. get_self() returned the same object");
  end

endmodule
```

EXPECTED OUTPUT

```
1. after new      : addr=11 data=22
2. after bad_set  : addr=11
3. after good_set : addr=99
4. get_self() returned the same object
```

Line by line

Line	Explanation
`this.addr = addr;`	Left side: the property of the current object. Right side: the constructor argument.
`addr = addr;` in `bad_set`	Both names resolve to the argument. The property keeps its old value, and there is no compiler error. Some tools give a warning.
Output line 2	`addr` is still 11: the bug is silent.
`return this;`	The method hands back the handle of the object it was called on.
`q == p`	Both handles refer to one object.

> **Interview trap:** "What does `addr = addr;` do in a constructor whose argument is called `addr`?" Nothing useful: it assigns the argument to itself. The class property is not changed. The fix is `this.addr = addr;`.

> **Verification application:** `this` is used when a component registers itself with another, for example a monitor that passes `this` to a callback object, and in UVM where a child component is created with `new(name, this)` so that the child knows its parent.

Interview questions

Q1. What is `this`? A predefined handle that refers to the current object, that is, the object on which the method was called.

Q2. When must `this` be used? When a method argument or local variable has the same name as a class property and you need to refer to the property; and when the object needs to pass or return its own handle.

Q3. Can `this` be used in a static method? No. A static method has no current object.

Q4. Can `this` be used outside a class? No. It is only meaningful inside non-static class methods (and in constraints and property initialisers of the class).

Part 2: Class Members in Depth

2.1 The Scope Resolution Operator (::)

In plain English

The dot operator reaches inside an **object**: `p.addr` means "the `addr` of the object that `p` refers to". The double-colon operator reaches inside a **named scope**, such as a class or a package, without needing any object: `packet::count` means "the item called `count` that belongs to the class `packet` itself".

Technical definition

The scope resolution operator `::` identifies a name within a specified scope. The left operand is the name of a class, a package, a covergroup type or a typedef name; the right operand is an item declared in that scope.

Where it is used

Use	Example	Covered in
Static property of a class	`packet::count`	Section 2.2
Static method of a class	`id_gen::get_id()`	Section 2.3
Type, enum label, `parameter` or `localparam` declared inside a class	`packet::dir_e`, `packet::WRITE`, `packet::HEADER_BYTES`	This section
Item in a package	`alu_pkg::alu_op_e`, `import alu_pkg::*;`	Module 11
Body of an `extern` method written outside the class	`function void packet::print();`	Section 2.4
Member of a parameterized class specialisation	`stack #(int)::depth`	Module 11
Base-class member from a derived class	`base_txn::MAX` (for calling an overridden base *method* use `super.`)	Part 4
Disambiguation in inline constraints	`local::addr`	Module 5
Compilation-unit scope	`$unit::name`	Module 11

Dot versus double colon

	`.` (dot)	`::` (scope resolution)
Left operand	A handle (an object), an instance, a struct variable	A class name, package name or type name
Needs an object to exist	Yes (for non-static members)	No
Reaches	Members of one particular object	Items that belong to the scope itself: static members, types, parameters, package items

	`.` (dot)	`::` (scope resolution)
Example	`p.addr`	`packet::count`

Static members may be reached either way (`p.count` or `packet::count`), but `::` is the clearer style because it shows that the member belongs to the class.

Complete example

tb_scope_resolution.sv

```
package cfg_pkg;
  parameter int MAX_LEN = 16;
  typedef enum {LOW, HIGH} level_e;
endpackage

module tb_scope_resolution;

  class packet;
    typedef enum {READ, WRITE} dir_e;        // a type declared inside the class
    localparam int HEADER_BYTES = 2;         // a constant declared inside the class
    static int count = 0;                    // a static property
    dir_e dir;

    static function int get_count();         // a static method
      return count;
    endfunction
  endclass

  packet::dir_e    d;                        // type from the class scope
  cfg_pkg::level_e lvl;                      // type from the package scope

  initial begin
    d   = packet::WRITE;                     // enum label from the class scope
    lvl = cfg_pkg::HIGH;                     // enum label from the package scope
    packet::count = 3;                       // static property: no object exists

    $display("%s %s %0d %0d %0d",
             d.name(), lvl.name(), packet::HEADER_BYTES, cfg_pkg::MAX_LEN,
             packet::get_count());
  end

endmodule
```

EXPECTED OUTPUT

```
WRITE HIGH 2 16 3
```

Line by line

Line	Explanation
`typedef enum {READ, WRITE} dir_e;` inside the class	The type and its labels live in the scope of the class. Outside the class they are `packet::dir_e`, `packet::READ`, `packet::WRITE`. This avoids name clashes between classes.
`localparam int HEADER_BYTES = 2;`	A class may declare constants.
`packet::dir_e d;`	A module-level variable whose type is taken from the class.
`packet::count = 3;`	No `packet` object has been created anywhere in this example. Static members exist without objects.
`cfg_pkg::MAX_LEN`	The same operator selects an item from a package without an `import`.

> **Common mistake:** Using the dot with a class name (`packet.count`) or the double colon with a handle (`p::count`). The dot follows a handle; the double colon follows a class or package name.

Interview questions

Q1. What is the scope resolution operator and where is it used? `::` selects a name inside a named scope. It is used to access static properties and methods of a class, types and parameters declared in a class, items in a package, and to define `extern` methods outside the class body.

Q2. What is the difference between `.` and `::`? The dot accesses a member of a specific object through a handle. The double colon accesses an item that belongs to a class or package as a whole and needs no object.

Q3. How do you refer to an enum declared inside a class from outside it? With the class name as prefix: `class_name::enum_type` for the type and `class_name::LABEL` for a label.

2.2 Static Properties

In plain English

Normal properties belong to each object: every packet has its own address. A **static** property belongs to the class as a whole: there is exactly one copy, shared by every object of the class. If one object changes it, all objects see the new value. It is like a notice board in the lobby of a block of flats: each flat has its own furniture, but there is one notice board for everybody.

Technical definition

A property declared with the keyword `static` has a single storage location associated with the class, not with any object. It is created and initialised once, at the start of simulation (before any `initial` block runs), and exists even if no object of the class is ever created. It can be accessed with the class scope operator (`class_name::property`) or through any handle of the class.

Why it is needed

Requirement	Solution with a static property
Count how many objects (transactions) have been created	`static int count;` incremented in `new()`
Give every object a unique ID	Copy the counter into a per-object `id` in `new()`
Share one configuration object, one virtual interface or one mailbox between all objects of a class	`static cfg_t cfg;`
Keep statistics for the whole class (total errors, total bytes)	`static int error_count;`
A value that all objects must agree on, without using a global variable	Static property, which is a global variable *scoped inside the class*

How it works internally

```
  class packet;
    static int count;      // one copy, belongs to the class
    int        id;         // one copy per object
  endclass

                        +----------------------+
    class packet        |  count = 3           |   static storage: exists from time 0,
                        +----------------------+   exists with zero objects
                           ^        ^        ^
                           |        |        |      every object reaches the same count
           +---------------+   +----+----+   +---------------+
           |                   |         |                    |
     +-----------+       +-----------+       +-----------+
p1 |  id = 1   |    p2 |  id = 2   |    p3 |  id = 3   |    one id per object
     +-----------+       +-----------+       +-----------+
```

Syntax

```
class packet;
  static int count = 0;          // static property, initialised once
  int        id;                 // ordinary (instance) property
endclass

packet::count = 5;               // access through the class name (preferred)
p1.count      = 5;               // access through a handle (legal, same variable)
```

Complete example

tb_static_property.sv

```
module tb_static_property;

  class packet;
    static int count = 0;        // shared by all packet objects
    int        id;               // unique to each object

    function new();
      count++;
      id = count;
    endfunction
  endclass

  packet p1, p2, p3;

  initial begin
    $display("1. before any object exists: count=%0d", packet::count);

    p1 = new();
    p2 = new();
    p3 = new();
    $display("2. ids: p1=%0d p2=%0d p3=%0d", p1.id, p2.id, p3.id);
    $display("3. count seen from: class=%0d p1=%0d p3=%0d",
             packet::count, p1.count, p3.count);

    p2.count = 10;               // written through one handle...
    $display("4. after p2.count = 10: class=%0d p1=%0d", packet::count, p1.count);
  end

endmodule
```

EXPECTED OUTPUT

```
1. before any object exists: count=0
2. ids: p1=1 p2=2 p3=3
3. count seen from: class=3 p1=3 p3=3
4. after p2.count = 10: class=10 p1=10
```

Line by line

Line	Explanation
`static int count = 0;`	One variable for the whole class, initialised once before time 0. The initialiser is **not** re-applied when objects are created.
`count++; id = count;`	Each construction increments the shared counter and copies it to the object's private `id`.
Output 1	The static property is readable before any object exists.
Output 2	Each object kept the value of the counter at the time it was built.
Output 3	Whichever way it is accessed, it is the same variable.
Output 4	A write through `p2` is seen by the class and by `p1`.

Static versus instance property

Property	Instance (non-static) property	Static property
Belongs to	Each object	The class
Number of copies	One per object	One in total
Memory allocated	When `new()` is called	At the start of simulation
Exists without an object	No	Yes
Accessed with	`handle.name`	`class_name::name` or `handle.name`
Access through a null handle	Fatal error	Legal (the handle's type is enough to find the class)
Declaration initialiser applied	For each new object	Once
Can be `rand`	Yes	Yes, but then every `randomize()` call changes the one shared value
Typical use	Transaction fields	Counters, shared configuration, shared resources

> **Interview trap:** "If I create ten objects of a class with a static property, how many copies of the property exist?" One. "And if I create zero objects?" Still one.

> **Common mistake:** Expecting `static int count = 0;` to be reset to 0 in every new object. It is initialised once. If the constructor needs a fresh value per object, the property must not be static.

> **Verification application:** The classic use is a transaction counter: `static int unsigned next_id;` in the transaction class, with `id = next_id++`-style logic in the constructor, so that every transaction in the log has a unique number that can be traced from generator to scoreboard.

Interview questions

Q1. What is a static property? A property declared with `static`. It has one copy shared by all objects of the class, exists without any object, and can be accessed with `class_name::property`.

Q2. When is a static property initialised? Once, at the start of simulation, before any procedural block executes.

Q3. How do you count the number of objects created from a class? Declare `static int count;` and increment it in the constructor.

Q4. Can a static property be accessed through a null handle? Yes. Static members do not need an object, so the access is legal even when the handle is `null`.

Q5. What is the difference between a static property and a global variable? Functionally both have one copy for the whole simulation. A static property is scoped inside its class, so it does not pollute the global name space, can be made `local` or `protected`, and is clearly associated with the class that uses it.

2.3 Static Methods

In plain English

A static method is a method that belongs to the class, not to an object. You call it with the class name, without creating anything. Because it is not called on an object, it has no "my own properties" to work with: it can use only static properties and other static methods.

Technical definition

A method declared with `static` before the keyword `function` or `task` is a static method. It:

- can be called without an object: `class_name::method()`;
- has **no `this` handle**;
- can access only **static** properties and call only **static** methods of its class directly (it can still use non-static members of an object whose handle it is given as an argument or creates itself);
- **cannot be `virtual`**.

Why it is needed

- To read or modify static properties that are `local` or `protected` without needing an object.
- For utility functions that logically belong to a class but need no object state (conversions, calculations, name look-ups).
- For "factory" and "singleton" patterns: a static method is the only way to obtain an object when the constructor is hidden. UVM uses this heavily (`my_class::type_id::create(...)`, `uvm_config_db#(T)::get(...)` are static method calls).

Syntax

```
class id_gen;
  local static int next_id = 100;

  static function int get_id();     // static METHOD
    int id;
    id = next_id;
    next_id++;
    return id;
  endfunction
endclass

x = id_gen::get_id();               // no object needed
```

Complete example

tb_static_method.sv

```
module tb_static_method;

  class id_gen;
    local static int next_id = 100;       // hidden shared state
    int              calls;               // an ordinary, per-object property

    static function int get_id();
      int id;
      id = next_id;
      next_id++;
      // calls++;                         // ILLEGAL: non-static property in a static method
      // this.calls = 0;                  // ILLEGAL: no "this" in a static method
      return id;
    endfunction

    static function void reset(int start = 100);
      next_id = start;
    endfunction
  endclass

  id_gen g;                               // never constructed in this example

  initial begin
    $display("1. %0d", id_gen::get_id());
    $display("2. %0d", id_gen::get_id());
    $display("3. %0d", g.get_id());       // legal even though g is null
    id_gen::reset(500);
    $display("4. %0d", id_gen::get_id());
  end

endmodule
```

EXPECTED OUTPUT

```
1. 100
2. 101
3. 102
4. 500
```

Line by line

Line	Explanation
`local static int next_id = 100;`	Shared state that nobody outside the class can touch directly.

Line	Explanation
`static function int get_id();`	Returns the current value and advances the counter.
The two commented lines	A static method has no object, so it cannot use a non-static property or `this`. Both would be compile errors.
`id_gen::get_id()`	Called through the class name. No `id_gen` object exists.
`g.get_id()`	A static method may also be called through a handle, even a null one, because only the handle's type is used. Prefer the `::` form.
`id_gen::reset(500);`	Another static method modifying the hidden state.

Static method versus static lifetime

The keyword `static` can appear in two different positions in a method declaration, with two different meanings. Interviewers like this question.

Declaration	Meaning	Legal in a class?
`static function int f();`	**Static method**: belongs to the class, no `this`, called with `::`. Its arguments and local variables still have *automatic* lifetime	Yes
`function static int f();`	Function with **static lifetime**: its arguments and local variables are allocated once and shared by all calls (Module 3)	**No.** Class methods must be automatic
`function automatic int f();`	Automatic lifetime, stated explicitly	Yes (it is already the default in a class)

Static versus non-static method

Property	Non-static method	Static method
Called on	An object: `h.method()`	The class: `class_name::method()` (or through a handle)
`this` available	Yes	No
Can access non-static properties	Yes	No (only via an explicit handle to some object)
Can access static properties	Yes	Yes
Can call non-static methods	Yes	No (only via an explicit handle)
Can be `virtual`	Yes	No
Works with a null handle	No, fatal error	Yes

Interview trap: "Can a static method call a non-static method?" Not directly, because it has no object. It can if it has a handle to an object: `static function void show(packet p); p.print(); endfunction` is legal.

Interview trap: "Can a non-static method access static properties?" Yes. The restriction applies only in the other direction.

Interview questions

Q1. What is a static method? A method declared with `static` before `function` / `task`. It belongs to the class, can be called without an object using `::`, has no `this`, and can directly access only static members.

Q2. Why can a static method not access non-static properties? Non-static properties exist only inside objects. A static method is not invoked on an object, so there is no object whose properties it could mean.

Q3. What is the difference between `static function` and `function static`? `static function` declares a static method (class-level, no `this`). `function static` declares static *lifetime* for the arguments and local variables, which is not permitted for class methods.

Q4. Can a static method be virtual? No. Virtual dispatch depends on the type of an object, and a static method has no object.

Q5. Give a practical use of static methods. Unique-ID generators, object counters with a `get_count()` accessor, singleton `get()` methods for a shared configuration, and factory `create()` methods as in UVM.

2.4 Extern Methods

In plain English

A class with many long methods is hard to read: you cannot see at a glance what the class offers. With `extern` you write only the *prototype* of a method (its name, return type and arguments) inside the class, and put the body somewhere else, after the class. The class declaration then reads like a table of contents.

Technical definition

A method prototype declared with the keyword `extern` inside a class is an **out-of-block declaration**. Its body is defined outside the class body, in the same scope as the class, with the method name qualified by the class name and `::`.

Syntax

```
class packet;
  extern function new(bit [7:0] addr = 0);             // prototypes only
  extern function void print(string tag = "");
  extern task drive(int delay);
endclass

function packet::new(bit [7:0] addr);                  // bodies, outside the class
  this.addr = addr;
endfunction

function void packet::print(string tag);
  ...
endfunction

task packet::drive(int delay);
  ...
endtask
```

Rules

Rule	Detail
The body must match the prototype	Same return type, same argument names, types and directions
Qualifiers go in the prototype	`virtual`, `static`, `local`, `protected`, `pure` are written with the `extern` prototype, not repeated on the body
Default argument values	Written in the prototype. They may be left out of the body; if they are repeated there, they must be identical
Return type declared inside the class	In the body it must be qualified, because the return type is written before `packet::` puts the class in scope: `function packet::dir_e packet::get_dir();`
Location of the body	In the same scope (package, module, file) that contains the class declaration
Inside the body	Properties and `this` are used exactly as in an inline method

Complete example

tb_extern.sv

```
module tb_extern;

  class packet;
    bit [7:0] addr;
    bit [7:0] data;

    extern function new(bit [7:0] addr = 8'h00, bit [7:0] data = 8'h00);
    extern function void print(string tag = "");
    extern function bit [7:0] checksum();
  endclass

  function packet::new(bit [7:0] addr, bit [7:0] data);
    this.addr = addr;
    this.data = data;
  endfunction

  function void packet::print(string tag);
    $display("[%s] addr=%h data=%h checksum=%h", tag, addr, data, checksum());
  endfunction

  function bit [7:0] packet::checksum();
    return addr ^ data;
  endfunction

  packet p;

  initial begin
    p = new(8'h0F, 8'hF1);
    p.print("p");
    p = new();
    p.print("default");
  end

endmodule
```

EXPECTED OUTPUT

```
[p] addr=0f data=f1 checksum=fe
[default] addr=00 data=00 checksum=00
```

Line by line

Line	Explanation
`extern function new(...)`	Even the constructor can be declared `extern`.
The class body	Three lines tell a reader everything the class can do.
`function packet::new(bit [7:0] addr, bit [7:0] data);`	The body, written after the class. The default values come from the prototype.
`function void packet::print(string tag);`	`packet::` tells the compiler which class the body belongs to.
`checksum()` inside `print`	An extern method calls other methods and uses properties like any other method.
`8'h0F ^ 8'hF1`	`0000_1111` XOR `1111_0001` = `1111_1110` = `FE`.

> **Verification application:** UVM-style code bases commonly write every method as `extern`, keeping the class declaration short, and put the bodies below the class in the same file. In a package, the class files are `` `include ``d into the package, so the extern bodies are in the package scope as required.

> **Common mistake:** Forgetting `class_name::` on the body. The compiler then sees an ordinary stand-alone function with the same name, and reports that the extern method has no body.

Interview questions

Q1. What is an extern method? A method whose prototype is declared inside the class with `extern` and whose body is written outside the class using `class_name::method_name`.

Q2. Why use extern methods? Readability and maintainability: the class declaration shows only the interface, and the implementations are separated from it.

Q3. Where do the qualifiers `virtual`, `local` and `static` go? In the prototype inside the class, together with `extern`. They are not repeated on the out-of-block body.

2.5 Constant Properties (const)

In plain English

Some values must never change after they have been set: the ID of a transaction, the maximum packet length. Marking such a property `const` makes the compiler reject any later assignment to it.

Technical definition

A class property declared with `const` is read-only. There are two kinds, distinguished by where the value is given.

Kind	Declaration	Where the value is assigned	Same value in every object?
Global constant	`const int MAX_LEN = 64;` (initialiser in the declaration)	In the declaration only; it cannot be assigned anywhere else, not even in `new`	Yes

Kind	Declaration	Where the value is assigned	Same value in every object?
Instance constant	`const int id;` (no initialiser)	Exactly once, in the constructor `new`	No, each object may get a different value

A global constant has the same value in all objects, so it is usually also declared `static` to store it once: `static const int MAX_LEN = 64;`. An instance constant cannot be `static`, because it is assigned in the constructor.

Complete example

tb_const.sv

```
module tb_const;

  class packet;
    static const int MAX_LEN = 64;      // global constant: value fixed in the declaration
    const int        id;                // instance constant: value fixed in new()
    bit [7:0]        data;

    function new(int id);
      this.id = id;                     // the one permitted assignment
    endfunction
  endclass

  packet p1, p2;

  initial begin
    p1 = new(1);
    p2 = new(2);
    $display("p1.id=%0d p2.id=%0d MAX_LEN=%0d", p1.id, p2.id, packet::MAX_LEN);

    p1.data = 8'h5A;                    // ordinary property: writable
    // p1.id = 7;                       // ILLEGAL: assignment to a const property
    // packet::MAX_LEN = 128;           // ILLEGAL: assignment to a const property
  end

endmodule
```

EXPECTED OUTPUT

```
p1.id=1 p2.id=2 MAX_LEN=64
```

const compared with related constructs

Construct	Value fixed at	Can differ per object	Scope
`parameter` / `localparam` in a class	Compile / elaboration time	No	The class (accessed with `::`)
Global `const` property	Compile time (declaration)	No	The class or object
Instance `const` property	Run time, in the constructor	Yes	The object
`` `define `` macro	Text substitution before compilation	No	Global, no scope

> **Interview trap:** "Can a `const` property be assigned in the constructor?" Only if its declaration has no initialiser (an instance constant). A `const` with an initialiser cannot be assigned anywhere.

> **Note:** `const` applied to a *handle* makes the handle constant, not the object: the handle cannot be made to refer to another object, but the properties of the object it refers to can still be modified.

Interview questions

Q1. What are the two kinds of constant class property? Global constants, initialised in the declaration and never assignable; and instance constants, declared without an initialiser and assigned exactly once in the constructor.

Q2. What is the difference between `const` and `localparam` in a class? A `localparam` is an elaboration-time constant, identical for all objects, usable where a constant expression is required (for example as an array dimension). An instance `const` is set at run time in the constructor and may differ for each object.

2.6 Local, Protected and Public Access

In plain English

By default anybody who holds a handle can read and write every property of the object. That is convenient and dangerous: any code anywhere in the testbench can put an object into an inconsistent state. Access qualifiers let the author of a class decide which members are for everybody and which are private details.

Think of a cash machine. The public part is the keypad and the screen. The cash box inside is not accessible; you can only change its contents by using the keypad, and the machine checks every request.

Technical definition

Qualifier	Keyword	Accessible from
Public	(none; it is the default)	Anywhere a handle to the object is available
Protected	`protected`	Inside the class itself and inside classes derived from it
Local	`local`	Inside the class itself only. Not in derived classes, not from outside

The qualifiers apply to properties and to methods. They are checked at **compile time**. SystemVerilog has no `public` or `private` keyword: public is expressed by writing nothing, and `local` is SystemVerilog's word for what C++ and Java call private.

Access matrix

Where the access is written	Public	`protected`	`local`
A method of the same class	Yes	Yes	Yes
A method of a derived class	Yes	Yes	**No**
Outside the class (a module, another unrelated class)	Yes	**No**	**No**

One more rule: access control is per **class**, not per object. A method of class `account` may access the `local` members of *any* `account` object it has a handle to, not only those of `this`. This is what makes `compare(rhs)` and `copy()` methods possible.

Syntax

```
class account;
  local     int balance;       // only methods of account
  protected int limit;         // methods of account and of its subclasses
  string    owner;             // public

  local function void audit(); // methods can be qualified too
    ...
  endfunction
endclass
```

Complete example

This example uses a derived class (`extends`) to show `protected`. Inheritance itself is explained in Part 4; for now read `class premium_account extends account` as "a `premium_account` has everything an `account` has, plus more".

tb_access.sv

```
module tb_access;

  class account;
    local     int balance;                // hidden
    protected int limit = 1000;           // visible to subclasses
    string    owner;                      // public

    function new(string owner, int balance);
      this.owner   = owner;
      this.balance = balance;
    endfunction

    function void deposit(int amount);
      if (amount > 0) balance += amount;   // the class guards its own data
    endfunction

    function bit withdraw(int amount);
      if (amount <= 0 || amount > balance) return 0;
      balance -= amount;
      return 1;
    endfunction

    function int get_balance();
      return balance;
    endfunction

    function bit richer_than(account other);
      return balance > other.balance;      // local member of ANOTHER account object: legal
    endfunction
  endclass

  class premium_account extends account;
    function new(string owner, int balance);
      super.new(owner, balance);
      limit = 5000;                        // protected member of the base class: legal
      // balance = 0;                      // ILLEGAL: balance is local to account
    endfunction

    function void show();
      $display("%s: limit=%0d balance=%0d", owner, limit, get_balance());
    endfunction
  endclass

  account         a;
  premium_account b;
  bit             ok;

  initial begin
    a = new("asha", 100);
```

```
    a.deposit(50);
    $display("1. balance=%0d", a.get_balance());
    ok = a.withdraw(500);
    $display("2. withdraw 500 ok? %0d   balance=%0d", ok, a.get_balance());
    ok = a.withdraw(100);
    $display("3. withdraw 100 ok? %0d   balance=%0d", ok, a.get_balance());

    b = new("ben", 900);
    b.show();
    $display("4. ben richer than asha? %0d", b.richer_than(a));

    a.owner = "asha k";                  // public: legal
    // a.balance = 1_000_000;            // ILLEGAL: local
    // a.limit   = 0;                    // ILLEGAL: protected
  end

endmodule
```

EXPECTED OUTPUT

```
1. balance=150
2. withdraw 500 ok? 0   balance=150
3. withdraw 100 ok? 1   balance=50
ben: limit=5000 balance=900
4. ben richer than asha? 1
```

Line by line

Line	Explanation
`local int balance;`	Can be changed only by `deposit` and `withdraw`, which enforce the rules (positive amounts, no overdraft).
`protected int limit = 1000;`	Hidden from outside code, but a subclass may adjust it.
`return balance > other.balance;`	`other` is a different object, but it is of the same class, so its local member is accessible.
`limit = 5000;` in `premium_account`	A derived class may access protected members of its base class.
`// balance = 0;` in `premium_account`	A derived class may **not** access local members of its base class.
`get_balance()` in `show()`	The subclass must use the public accessor, like everybody else.
`ok = a.withdraw(500);`	The result is stored first and printed afterwards. Calling a method that changes the object inside the argument list of `$display`, next to another call that reads the object, would depend on the order in which the simulator evaluates the arguments.
Output 2	500 exceeds the balance of 150, so `withdraw` returns 0 and nothing changes.
The last two commented lines	Outside code can reach neither local nor protected members.

Comparison table

Aspect	Public	`protected`	`local`
Keyword	None	`protected`	`local`
Same class	Yes	Yes	Yes

Aspect	Public	`protected`	`local`
Derived class	Yes	Yes	No
Outside	Yes	No	No
Equivalent in C++/Java	`public`	`protected`	`private`
Typical use	Transaction fields that are randomized and read everywhere; the methods that form the class's interface	Internal helpers and state that subclasses are expected to extend	Implementation details nobody else should depend on

> **Interview trap:** "What is the default access of a class member in SystemVerilog?" Public. (In C++ the default for a `class` is private, which is why this is asked.)

> **Interview trap:** "Can a `local` member be accessed through a different object of the same class?" Yes. Access is checked against the class in which the code is written, not against the object.

> **Common mistake:** Making transaction fields `local` and then being unable to constrain them in `randomize() with {...}` from a test or read them in the scoreboard. In verification, transaction data fields are normally public; `local` and `protected` are used mainly for the internal state of components.

> **Note:** A constructor can itself be declared `local` or `protected` (`local function new();`). Then code outside the class cannot call `new` for it, and objects can be obtained only through a static method of the class. This is the basis of the singleton pattern (Exercise 6 in Part 6).

Interview questions

Q1. What are the access qualifiers in SystemVerilog? `local`, `protected`, and public (the default, with no keyword).

Q2. What is the difference between `local` and `protected`? A `local` member is accessible only within the class that declares it. A `protected` member is accessible within that class and within all classes derived from it. Neither is accessible from outside.

Q3. When are access violations detected? At compile time.

Q4. How do you read a local property from outside the class? Through a public method of the class (a "getter"), such as `get_balance()`.

Q5. Why hide data at all? So that the object's state can change only through methods that keep it consistent, and so that the internal representation can be changed later without breaking code that uses the class. This is encapsulation (Section 4.2).

2.7 Forward Declaration with typedef class

The problem

The compiler reads a file from top to bottom. A class name must be known before it is used as the type of a handle. When two classes refer to each other, there is no order in which both are declared before use:

```
class demo_monitor;
  demo_driver drv;      // ERROR: demo_driver is not known yet
endclass

class demo_driver;
  demo_monitor mon;     // fine: demo_monitor was declared above
endclass
```

The solution

A **forward declaration** tells the compiler "this name is a class; its definition comes later":

```
typedef class demo_driver;
```

After that line, handles of type `demo_driver` may be declared. The full class definition must appear later in the same scope.

Complete example

tb_forward_typedef.sv

```
module tb_forward_typedef;

  typedef class demo_driver;          // forward declaration

  class demo_monitor;
    string      name = "monitor";
    demo_driver drv;                  // legal because of the typedef above
  endclass

  class demo_driver;
    string       name = "driver";
    demo_monitor mon;
  endclass

  demo_monitor m;
  demo_driver  d;

  initial begin
    m = new();
    d = new();
    m.drv = d;                        // connect the two objects to each other
    d.mon = m;
    $display("%s knows %s", m.name, m.drv.name);
    $display("%s knows %s", d.name, d.mon.name);
  end

endmodule
```

EXPECTED OUTPUT

```
monitor knows driver
driver knows monitor
```

When a forward declaration is needed

Situation	Needed?
Two classes hold handles of each other (circular reference)	Yes
A class is used as a handle type before its definition appears in the file or package	Yes
A class holds a handle of its own type (`node next;` inside `class node`)	No, the class name is already known inside its own body
The forward-declared class is only used as a handle type before its definition	Fine
You try to call `new()` on it, access its members, or extend it before the definition	Not allowed: the compiler needs the full definition for these

> **Verification application:** In UVM and in layered testbenches, circular references are common: an environment holds its agents and each agent may hold a handle back to a shared configuration or to its parent. Packages therefore often start with a list of `typedef class` lines.

Interview questions

Q1. What is a forward declaration and why is it needed? `typedef class name;` announces that `name` is a class defined later. It is needed when a class is referenced before its definition, most often when two classes contain handles to each other.

Q2. Can you create an object of a forward-declared class before its definition? No. Only handle declarations are possible until the compiler has seen the full definition.

2.8 Object Lifetime and Garbage Collection

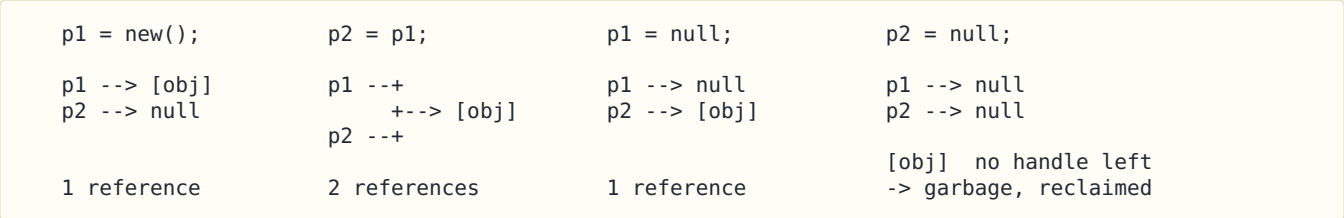
In plain English

You create objects with `new()`. You never delete them. The simulator watches which objects can still be reached through some handle. When an object can no longer be reached by anybody, it is useless, and the simulator takes its memory back automatically. This is called **garbage collection**.

Technical definition

- An object's lifetime begins when `new()` returns.
- An object remains alive as long as it is **reachable**: at least one live handle refers to it, directly or through a chain of other reachable objects (a handle in a variable, a class property, an array or queue element, a mailbox entry, an argument or local variable of a method that has not finished, including methods suspended in running threads).
- When an object becomes unreachable, the simulator may reclaim its memory at any time. SystemVerilog provides automatic memory management; there is no `delete` or `free` for objects and there are **no destructors**.

How it works



Ways in which an object loses a reference:

Action	Example
The handle is set to `null`	`p = null;`
The handle is assigned another object	`p = new();` or `p = other;`
The handle variable ceases to exist	A local variable of a task that returns
The container entry is removed	`q.pop_front()`, `q.delete()`, `mbx.get(t)` followed by `t` going out of scope
The object holding the handle becomes garbage itself	An environment object is dropped, and with it everything only it referred to

Common pattern: one new object per loop iteration

```
task alu_generator::run();
  alu_txn txn;
  repeat (num_txns) begin
    txn = new();                  // a NEW object every iteration
    void'(txn.randomize());
    gen2drv_mbx.put(txn);         // the mailbox now also refers to the object
  end                             // next iteration: txn is re-assigned, but the old
endtask                           // object stays alive because the mailbox holds it
```

Each object lives until the driver has taken it out of the mailbox and no component holds its handle any longer. Nobody frees anything explicitly. The opposite mistake, creating the object once outside the loop, is shown in Section 3.1.

Facts worth knowing

Fact	Detail
No destructor	There is no method that runs when an object is reclaimed. Clean-up that matters (closing files, final reports) must be called explicitly
You cannot force collection	Setting handles to `null` only makes an object *eligible*
Circular references	Two objects that refer only to each other, with no outside handle, are unreachable and are collected. The mechanism is defined by reachability, not by a simple reference count
Memory "leaks" still happen	If handles are kept forever in a queue, associative array or scoreboard that is never emptied, the objects are reachable and are never collected. Long simulations slow down and run out of memory this way
Threads keep objects alive	A task of an object that is still running in a `fork` branch keeps that object (through `this`) and its local handles alive

Fact	Detail
Static handles	An object referred to by a static property is alive for the whole simulation unless the property is cleared

Interview trap: "How do you delete an object in SystemVerilog?" You do not. You remove all references to it (for example by assigning `null`), and the simulator reclaims the memory automatically. There is no `delete` for objects and no destructor.

Verification application: A scoreboard that stores every expected transaction in a queue and never removes matched entries is the usual cause of steadily growing memory in long regressions. Remove items when they have been compared.

Interview questions

Q1. How is memory for objects released in SystemVerilog? Automatically, by garbage collection. When no handle refers to an object any more, the simulator reclaims its memory.

Q2. Does SystemVerilog have destructors? No.

Q3. `p1 = new(); p2 = p1; p1 = null;` Is the object destroyed? No. `p2` still refers to it. It becomes eligible for collection only when `p2` is also changed.

Q4. Can a class-based testbench have a memory leak? Not in the C sense of unreachable allocated memory, but effectively yes: objects kept in a growing queue, array or mailbox that is never drained stay reachable and are never freed.

Q5. What happens to objects created inside a task when the task returns? If the only handles to them were the task's local variables, they become unreachable and are collected. If a handle was stored elsewhere (returned, put in a mailbox, saved in a property), the object lives on.

Part 3: Handle Assignment and Object Copying

There are three different things people mean when they say "copy an object". Interviewers ask about them constantly, because confusing them produces bugs that are hard to see.

What you write	What is duplicated	Name
`p2 = p1;`	Only the handle. Still one object	Handle assignment
`p2 = new p1;`	The object, one level deep. Nested objects are shared	Shallow copy
`p2 = p1.copy();` (a method you write)	The object and every object nested inside it	Deep copy

3.1 Handle Assignment

In plain English

Assigning one handle to another does not copy the object. It copies the address. Afterwards two pieces of paper have the same address written on them, and there is still one house.

Technical definition

The assignment `h2 = h1;` between class handles copies the **reference**. Both handles then refer to the same object. Nothing is allocated and no property is copied. Any change made through one handle is visible through the other. This is completely different from assigning structs or arrays, where the data is copied.

How it works internally

```
packet p1, p2;
p1 = new();  p1.addr = 8'h10;        p2 = p1;                 p2.addr = 8'h99;

p1 --> [ addr = 10 ]                 p1 --+                   p1 --+
p2 --> null                               +--> [ addr = 10 ]       +--> [ addr = 99 ]
                                     p2 --+                   p2 --+

                                     one object, two handles  p1.addr is now 99 as well
```

Where handle assignment happens without an `=` sign

Handles are copied in all of these situations, and in every one of them the object is **not** duplicated:

Situation	Example
Passing an object as a task or function argument	`drv.send(txn);` The method receives a copy of the handle and works on the caller's object
Returning an object from a function	`return txn;`
Putting an object into a mailbox, queue or array	`gen2drv_mbx.put(txn);` `q.push_back(txn);`
Getting an object out of a mailbox	`mon2scb_mbx.get(txn);`

Situation	Example
Storing an object in a property of another object	`this.cfg = cfg;`

This is exactly why classes are efficient for passing transactions around a testbench: only a reference moves. It is also why the following bug is so common.

The classic bug: one object, put many times

tb_new_in_loop.sv

```
module tb_new_in_loop;

  class packet;
    bit [7:0] addr;
  endclass

  packet p;
  packet q_bad  [$];        // queues of handles (Module 2)
  packet q_good [$];

  initial begin
    // WRONG: the object is created once, outside the loop
    p = new();
    for (int i = 0; i < 3; i++) begin
      p.addr = i;
      q_bad.push_back(p);            // the same handle is stored three times
    end

    // RIGHT: a new object is created in every iteration
    for (int i = 0; i < 3; i++) begin
      p = new();
      p.addr = i;
      q_good.push_back(p);
    end

    foreach (q_bad[i])  $display("q_bad[%0d].addr  = %0d", i, q_bad[i].addr);
    foreach (q_good[i]) $display("q_good[%0d].addr = %0d", i, q_good[i].addr);
  end

endmodule
```

EXPECTED OUTPUT

```
q_bad[0].addr  = 2
q_bad[1].addr  = 2
q_bad[2].addr  = 2
q_good[0].addr = 0
q_good[1].addr = 1
q_good[2].addr = 2
```

Explanation

```
   q_bad                                q_good

   [0] --+                              [0] --> [ addr = 0 ]
   [1] --+--> [ addr = 2 ]              [1] --> [ addr = 1 ]
   [2] --+                              [2] --> [ addr = 2 ]

   three handles, ONE object            three handles, three objects
   (it holds the last value written)
```

In the first loop only one object ever exists. The queue receives the same reference three times, and each iteration overwrites the single `addr`. When the queue is read afterwards, all three entries show the final value, 2.

> **Verification application:** This is the most frequent beginner bug in a generator. If the generator creates the transaction once and then, in a loop, randomizes it and puts it into the mailbox, a driver that is slower than the generator sees the same (last) transaction repeated, and a scoreboard that stored "expected" handles finds they have all changed. The fix is `txn = new();` inside the loop, or putting a copy into the mailbox: `gen2drv_mbx.put(txn.copy());`.

Comparison with value types

Type of `a` and `b`	Effect of `b = a;`	Later change to `b` affects `a`?
`int`, `logic [7:0]`, `real`, `string`, enum	Value copied	No
`struct` (packed or unpacked)	All members copied	No
Fixed, dynamic, associative array, queue	All elements copied	No
Class handle	Reference copied	**Yes**, they share one object

Interview questions

Q1. What happens when one handle is assigned to another? The reference is copied. Both handles point to the same object; no new object is created.

Q2. An object is passed to a function and the function changes one of its properties. Does the caller see the change? Yes. The function received a copy of the handle, which refers to the caller's object. (This is true without `ref`. The `ref` keyword would only matter if the function assigned a *different object* to the handle and the caller needed to see that.)

Q3. A generator creates one transaction before a loop, randomizes it ten times and puts it in a mailbox each time. What does the driver get? Ten handles to the same object. Depending on timing it may see the same values repeatedly, typically those of the last randomization. A new object must be constructed for each transaction.

3.2 Shallow Copy

In plain English

A shallow copy makes a second object and copies every property of the first object into it. For ordinary data that gives an independent duplicate. But if one of the properties is a *handle* to another object, the handle is what gets copied. The two outer objects then share the same inner object.

Technical definition

The expression `new source_handle` (the keyword `new` followed by a handle, with **no parentheses**) allocates a new object of the same class and copies the value of every property from the source object into it:

- integral, `real`, `string`, enum and struct properties: the value is copied;
- array properties (fixed, dynamic, associative, queue): the whole array is copied, element by element;
- **class handle properties (and handles stored inside arrays): only the handle is copied**; the object it refers to is not duplicated.

The class constructor `new()` is **not executed** for a shallow copy.

Syntax

```
packet p1, p2;
p1 = new();        // normal construction: parentheses, constructor runs
p2 = new p1;       // shallow copy: no parentheses, constructor does NOT run
```

How it works internally

```
class header;  bit [7:0] id;                      endclass
class packet;  bit [7:0] data;  header hdr;       endclass

p1 = new();  p1.data = AA;  p1.hdr.id = 01;
p3 = new p1;                                      // shallow copy

     p1 --> +-------------+                 +-----------+
            | data = AA   |                 |  id = 01  |   ONE header object,
            | hdr  = *----|---------------> |           |   shared by both packets
            +-------------+        +------> +-----------+
                                   |
     p3 --> +-------------+        |
            | data = AA   |        |        data was duplicated:
            | hdr  = *----|--------+        p3.data = BB does not touch p1.data
            +-------------+                 hdr was only re-pointed:
                                            p3.hdr.id = 99 changes "p1.hdr.id" too
```

Complete example

tb_shallow_copy.sv

```
module tb_shallow_copy;

  class header;
    bit [7:0] id;
  endclass

  class packet;
    bit [7:0] data;
    header    hdr;

    function new();
      hdr = new();
      $display("   packet::new() executed");
    endfunction
  endclass

  packet p1, p2, p3;

  initial begin
    p1 = new();
    p1.data   = 8'hAA;
    p1.hdr.id = 8'h01;

    p2 = p1;            // handle assignment: no new object
    p3 = new p1;        // shallow copy: a new packet object, constructor not called

    p3.data   = 8'hBB;  // changes only p3
    p3.hdr.id = 8'h99;  // changes the header shared by p1 and p3

    $display("p1: data=%h hdr.id=%h", p1.data, p1.hdr.id);
    $display("p2: data=%h hdr.id=%h", p2.data, p2.hdr.id);
    $display("p3: data=%h hdr.id=%h", p3.data, p3.hdr.id);
    $display("p1 == p2: %0d   p1 == p3: %0d   p1.hdr == p3.hdr: %0d",
             p1 == p2, p1 == p3, p1.hdr == p3.hdr);
  end

endmodule
```

EXPECTED OUTPUT

```
   packet::new() executed
p1: data=aa hdr.id=99
p2: data=aa hdr.id=99
p3: data=bb hdr.id=99
p1 == p2: 1   p1 == p3: 0   p1.hdr == p3.hdr: 1
```

Line by line

Line	Explanation
`p1 = new();`	The constructor runs once (one message is printed) and creates the nested `header`.
`p2 = p1;`	Same object. `p1 == p2` is 1.
`p3 = new p1;`	A second `packet` object. The message is **not** printed again: the constructor does not run for a shallow copy. `p1 == p3` is 0.
`p3.data = 8'hBB;`	`data` is an ordinary property with its own storage in each object. `p1.data` stays `AA`.
`p3.hdr.id = 8'h99;`	`p3.hdr` and `p1.hdr` are the same reference. The single header object is changed, so `p1.hdr.id` also reads `99`.
`p1.hdr == p3.hdr`	1: proof that the nested object is shared.

What a shallow copy does and does not duplicate

Property kind	Duplicated by `new src`?
`int`, `bit [..]`, `logic [..]`, `real`, enum	Yes
`string`	Yes
Struct	Yes
Fixed array, dynamic array, queue, associative array of non-handle elements	Yes, all elements
Class handle	**No**: the handle is copied, the object is shared
Array or queue of class handles	The array is duplicated, but its elements are handles to the **same** objects
Static property	Not applicable: there is only one, belonging to the class
Per-object ID assigned in the constructor	Copied as-is, because the constructor does not run. The copy has the same ID as the original

Interview trap: "Does a shallow copy call the constructor?" No. `new p1` copies the property values directly. Anything the constructor normally does (incrementing a static counter, assigning a unique ID, printing a message, creating nested objects) does not happen for the copy.

Common mistake: Writing `p2 = new(p1);` when a shallow copy was intended. With parentheses this is an ordinary constructor call with `p1` as an argument, which is an error unless the constructor happens to accept a `packet` argument.

Common mistake: Assuming a shallow copy is "broken" for every class. If the class has no handle properties, a shallow copy is a complete, independent copy and is perfectly adequate.

Interview questions

Q1. What is a shallow copy? A copy made with `new handle;`. A new object is allocated and all properties are copied. Properties that are handles are copied as handles, so nested objects are shared between the original and the copy.

Q2. What is the syntax, and how does it differ from construction? `p2 = new p1;` with no parentheses. `p2 = new();` constructs a fresh object by running the constructor.

Q3. After a shallow copy, which changes to the copy affect the original? Changes to nested objects reached through handle properties. Changes to ordinary properties and to array properties do not.

Q4. Is a dynamic array property shared after a shallow copy? No. Arrays are values, not references; the copy gets its own array. Only class handles are shared.

3.3 Deep Copy

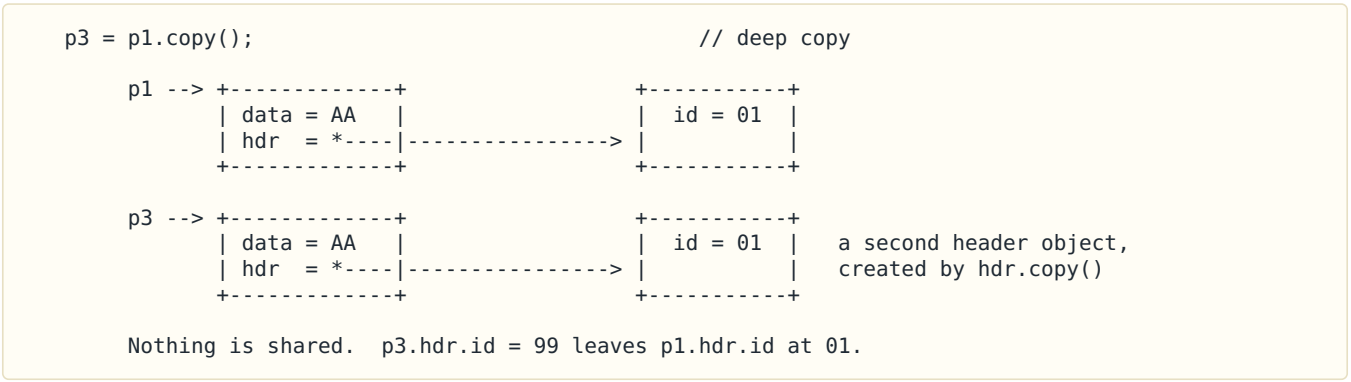
In plain English

A deep copy duplicates the object **and everything it refers to**, all the way down. The copy and the original have nothing in common afterwards. SystemVerilog has no built-in deep copy, because only the author of a class knows which nested objects should be duplicated. You write a `copy()` method yourself.

Technical definition

A deep copy is a user-written operation that creates a new object, copies all non-handle properties, and for every handle property creates a **new nested object** that is itself a deep copy of the original's nested object. The usual implementation is a `copy()` function in each class; the outer class's `copy()` calls the inner class's `copy()`.

How it works internally



Complete example

tb_deep_copy.sv

```
module tb_deep_copy;

  class header;
    bit [7:0] id;

    function header copy();
      header h = new();
      h.id = this.id;
      return h;
    endfunction
  endclass

  class packet;
    bit [7:0] data;
    header    hdr;

    function new();
      hdr = new();
    endfunction

    function packet copy();
      packet p = new();              // 1. a new packet (its constructor creates a header)
      p.data = this.data;            // 2. copy the ordinary properties
      p.hdr  = this.hdr.copy();      // 3. deep-copy the nested object
      return p;
    endfunction
  endclass
```

```
  packet p1, p3;

  initial begin
    p1 = new();
    p1.data   = 8'hAA;
    p1.hdr.id = 8'h01;

    p3 = p1.copy();                  // deep copy

    p3.data   = 8'hBB;
    p3.hdr.id = 8'h99;

    $display("p1: data=%h hdr.id=%h", p1.data, p1.hdr.id);
    $display("p3: data=%h hdr.id=%h", p3.data, p3.hdr.id);
    $display("p1 == p3: %0d   p1.hdr == p3.hdr: %0d", p1 == p3, p1.hdr == p3.hdr);
  end

endmodule
```

EXPECTED OUTPUT

```
p1: data=aa hdr.id=01
p3: data=bb hdr.id=99
p1 == p3: 0   p1.hdr == p3.hdr: 0
```

Line by line

Line	Explanation
`function header copy();`	The inner class knows how to copy itself.
`header h = new();`	A local handle with an initialiser. Class methods are automatic, so this runs on every call.
`function packet copy();`	Return type is the class itself.
`packet p = new();`	Creates the new outer object. Its constructor has already created a fresh `header`, which step 3 replaces.
`p.hdr = this.hdr.copy();`	The key line: instead of copying the handle, ask the nested object for a copy of itself.
Output	`p1` is untouched by the changes to `p3`. `p1.hdr == p3.hdr` is 0: two separate header objects.

If `hdr` could legitimately be `null`, the method must guard the call: `if (this.hdr != null) p.hdr = this.hdr.copy();`.

The three kinds of copy compared

Property	Handle assignment	Shallow copy	Deep copy
Syntax	`p2 = p1;`	`p2 = new p1;`	`p2 = p1.copy();` (user-written)
New outer object created	No	Yes	Yes
Ordinary properties	Shared (same object)	Duplicated	Duplicated
Nested objects (handle properties)	Shared	Shared	Duplicated
Constructor executed	No	No	Yes, as written in `copy()`
Built into the language	Yes	Yes	No

Property	Handle assignment	Shallow copy	Deep copy
Number of objects after copying a packet with one header	2 (1 packet, 1 header)	3 (2 packets, 1 header)	4 (2 packets, 2 headers)
Cost	Lowest	Low	Highest
Use when	You want another name for the same object, or to pass it on	The class has no handle properties, or sharing the nested objects is intended	The copy must be fully independent

Interview trap: "If a class contains only `int` and `bit` fields, is there a difference between a shallow and a deep copy?" No. The difference appears only when the class contains handles to other objects.

Common mistake: Writing a `copy()` method that does `p.hdr = this.hdr;`. That is a hand-written shallow copy. It looks like a deep copy from outside and behaves like a shallow one.

Common mistake: Adding a new property to a class and forgetting to add it to `copy()` (and to `compare()` and `print()`). The copy then silently has the default value for that field. UVM's field macros and `do_copy` convention exist to reduce this risk.

Verification application: A monitor that reuses one transaction object for every bus transfer must send a **copy** to the scoreboard; otherwise the scoreboard's stored handles all end up pointing at the object the monitor keeps overwriting. Equally, when one transaction is broadcast to a scoreboard and a coverage collector and either might modify it, each should receive its own copy.

Interview questions

Q1. What is a deep copy? A copy in which the object and all objects nested inside it are duplicated, so that the copy shares nothing with the original. It is implemented by a user-written method.

Q2. What is the difference between shallow and deep copy? A shallow copy (`new h`) duplicates only the top-level object; handle properties are copied as references, so nested objects are shared. A deep copy also duplicates the nested objects.

Q3. How do you implement a deep copy? Write a `copy()` function in each class. It constructs a new object, assigns all value properties, and for each handle property calls that object's own `copy()`.

Q4. Does SystemVerilog provide a built-in deep copy? No. Only handle assignment and shallow copy are built in.

3.4 Copy Constructors and Copy Methods

What "copy constructor" means in SystemVerilog

In C++ a *copy constructor* is a constructor that takes another object of the same class and initialises the new object from it; the language calls it automatically in many places. SystemVerilog has **no automatic copy**

constructor. The term is used loosely in interviews for "the code that makes a copy of an object". There are two idioms.

Idiom 1: a constructor that accepts a source object

```
class packet;
  bit [7:0] addr;
  bit [7:0] data;

  function new(packet src = null);
    if (src != null) begin
      this.addr = src.addr;
      this.data = src.data;
    end
  endfunction
endclass

packet p1 = new();        // src defaults to null: a fresh object
packet p2 = new(p1);      // "copy constructor": initialised from p1
```

Note the parentheses: `new(p1)` calls the constructor with an argument, while `new p1` is the built-in shallow copy.

Limitations: the single constructor now has to serve two purposes, its argument list is occupied by `src`, and it does not work well with inheritance, because the constructor cannot be virtual.

Idiom 2: a copy() method (the standard practice)

```
function packet copy();
  packet p = new();
  p.addr = this.addr;
  p.data = this.data;
  return p;
endfunction
```

This is the form used throughout these notes and in industry. A variation splits it in two, which becomes important with inheritance:

Method	Job
`copy()` or `clone()`	Creates a new object of the right class and returns it
`copy_from(src)` / `do_copy(src)`	Copies the fields of `src` into an *existing* object (`this`)

Section 5.6 shows why the split is needed when classes are extended.

Comparison

	`new(src)` constructor idiom	`copy()` method
Call	`p2 = new(p1);`	`p2 = p1.copy();`
Can be `virtual` (works through a base-class handle)	No	Yes
Leaves the constructor free for other arguments	No	Yes
Used by UVM	No	Yes (`copy` , `clone` , `do_copy`)

Interview questions

Q1. Does SystemVerilog have copy constructors? Not as a language feature. A constructor can be written to take a source object and copy its fields, but the usual solution is a user-defined `copy()` method.

Q2. What is the difference between `p2 = new p1;` and `p2 = new(p1);`? `new p1` is the built-in shallow copy; the constructor is not called. `new(p1)` calls the class constructor and passes `p1` as its argument; what happens depends entirely on how the constructor is written.

3.5 Comparing Objects

Handle comparison versus content comparison

Expression	Compares	True when
`p1 == p2`	The two handles	Both refer to the same object, or both are `null`
`p1.compare(p2)` (user-written)	The contents	The properties you chose to compare are equal

Two separately created objects with identical field values are **not** equal under `==`. A scoreboard that checks `if (actual == expected)` on transaction handles will report a mismatch for every transaction, or, worse, a match when both handles happen to refer to the same object.

Writing compare()

```
function bit compare(packet rhs);
  if (rhs == null) return 0;                    // nothing to compare with
  return (this.addr == rhs.addr) &&
         (this.data == rhs.data) &&
         this.hdr.compare(rhs.hdr);             // deep compare of the nested object
endfunction
```

Guidelines:

- Check `rhs` for `null` first.
- Compare four-state properties that carry DUT values with `===`, so that X or Z is a mismatch and not an "unknown" (Module 1, Section 4.5).
- Compare nested objects by calling their `compare()`, not by comparing the handles.
- Leave out fields that are not part of the functional content (IDs, timestamps), or the comparison will always fail.

3.6 The Complete alu_txn Class

Everything in Parts 1 to 3 comes together in the transaction class of the ALU testbench. This is the class used in all later modules. The fields are `rand` so that Module 5 can randomize them; in this module `rand` has no effect and can be ignored.

tb_alu_txn.sv

```
module tb_alu_txn;

  // In the real testbench this type comes from alu_pkg (Module 1).
  // It is repeated here so that the file runs on its own.
```

```
  typedef enum logic [2:0] {
    ALU_ADD, ALU_SUB, ALU_AND, ALU_OR, ALU_XOR, ALU_NOT, ALU_SHL, ALU_SHR
  } alu_op_e;

  class alu_txn;
    rand bit [7:0] a;
    rand bit [7:0] b;
    rand alu_op_e  op;
    logic [8:0]    result;       // filled in by the monitor from the DUT
    int unsigned   id;

    function void print(string tag = "");
      $display("[%s] id=%0d a=%0d b=%0d op=%s result=%0d",
               tag, id, a, b, op.name(), result);
    endfunction

    function alu_txn copy();
      alu_txn t = new();
      t.a      = this.a;
      t.b      = this.b;
      t.op     = this.op;
      t.result = this.result;
      t.id     = this.id;
      return t;
    endfunction

    function bit compare(alu_txn rhs);
      if (rhs == null) return 0;
      return (a == rhs.a) && (b == rhs.b) && (op == rhs.op) && (result === rhs.result);
    endfunction
  endclass

  alu_txn t1, t2;

  initial begin
    t1 = new();
    t1.id     = 1;
    t1.a      = 8'd200;
    t1.b      = 8'd100;
    t1.op     = ALU_ADD;
    t1.result = 9'd300;
    t1.print("t1");

    t2 = t1.copy();
    t2.print("t2");
    $display("same object? %0d   same contents? %0d", t1 == t2, t1.compare(t2));

    t2.b = 8'd101;
    $display("after t2.b = 101: same contents? %0d", t1.compare(t2));
    t1.print("t1");
  end

endmodule
```

EXPECTED OUTPUT

```
[t1] id=1 a=200 b=100 op=ALU_ADD result=300
[t2] id=1 a=200 b=100 op=ALU_ADD result=300
same object? 0   same contents? 1
after t2.b = 101: same contents? 0
[t1] id=1 a=200 b=100 op=ALU_ADD result=300
```

Line by line

Line	Explanation
`rand bit [7:0] a;`	A two-state random stimulus field (Module 1: `bit` for random data).
`logic [8:0] result;`	Four-state, because it will hold a value sampled from the DUT and must be able to show X. It is not `rand`: the testbench does not choose the result.

Line	Explanation
`int unsigned id;`	A serial number for tracing a transaction through the log.
`print(string tag = "")`	The tag lets each component label its own messages, for example `txn.print("DRV")`.
`copy()`	All fields are values, so there is nothing nested to deep-copy; a shallow copy `new t1` would give the same result. Writing `copy()` anyway keeps the interface uniform and ready for derived classes.
`compare(alu_txn rhs)`	`result` is compared with `===` so that an X from the DUT is a mismatch. `id` is deliberately not compared.
`t1 == t2`	0: two different objects.
`t1.compare(t2)`	1: the same contents.
`t2.b = 8'd101;`	Changes only the copy. The final `t1.print` proves the original is intact.

> **Key point:** Every transaction class should offer at least `print()`, `copy()` and `compare()`. They are what the driver, monitor and scoreboard need, and UVM standardises exactly these operations (`print`, `copy`/`clone`, `compare`).

Interview questions

Q1. How do you compare two objects? With a user-written `compare()` method that compares the relevant properties (and calls `compare()` on nested objects). The `==` operator on handles only tells you whether they refer to the same object.

Q2. Why is `result` declared `logic` and not `rand bit` in `alu_txn`? It is an output observed from the DUT, not a stimulus, so it is not randomized; and it must be four-state so that an X or Z driven by the DUT is preserved and detected in the comparison.

Q3. Why does `compare()` use `===` for `result`? With `==`, an X in the DUT result would make the comparison X, which is treated as false in one place and silently ignored in another, depending on how the code is written. `===` always returns 0 or 1 and treats X as a mismatch.

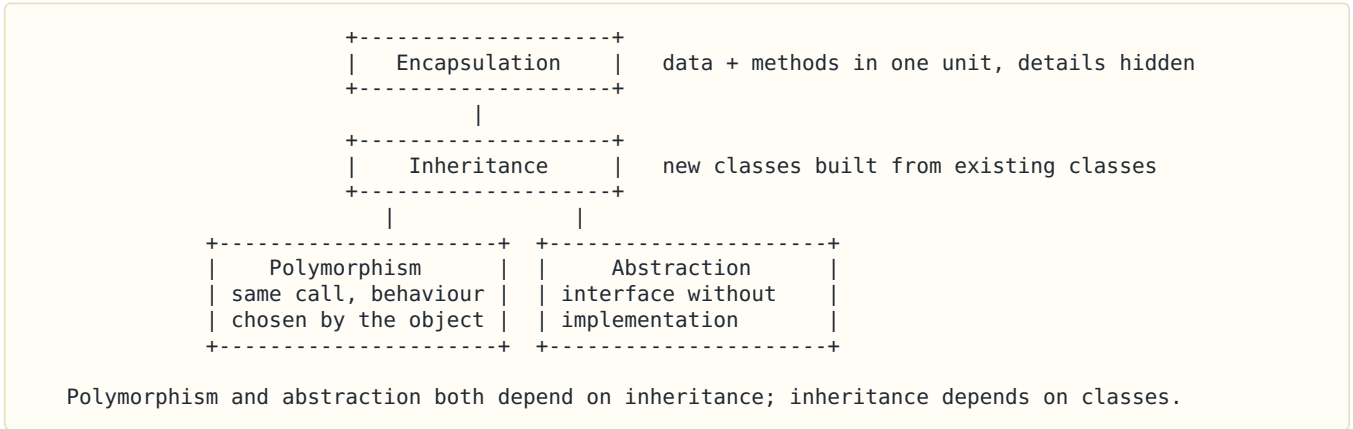
Q4. Why should a scoreboard store a copy of a transaction and not the handle it received? The sender may reuse and modify the same object later. If the scoreboard kept only the handle, its stored "expected" data would change underneath it.

Part 4: The OOP Pillars, Encapsulation and Inheritance

4.1 The Four Pillars of OOP

Object-oriented programming rests on four ideas. Interviewers expect you to name all four, define each in one or two sentences, and give a SystemVerilog example of each.

Pillar	One-sentence definition	SystemVerilog mechanism	Testbench example
Encapsulation	Keep data and the methods that operate on it together in one unit, and hide the internal details	`class`, `local`, `protected`	`alu_txn` holds its fields with `print`, `copy`, `compare`; a scoreboard hides its expected-queue behind `write()`
Inheritance	Build a new class from an existing one, reusing its members and adding or changing only what differs	`extends`, `super`	`class alu_arith_txn extends alu_txn` adds a constraint; `class smoke_test extends base_test`
Polymorphism	One handle type, many object types: the same method call does different things depending on the actual object	`virtual` methods, base-class handles	The driver calls `txn.print()` on an `alu_txn` handle and the derived class's version runs
Abstraction	Define *what* must be done without saying *how*; expose only the essential interface	`virtual class`, `pure virtual` methods	A base component declares `pure virtual task run();` and every component implements it



How to say it in an interview: "Encapsulation is bundling data and methods in a class and controlling access with local and protected. Inheritance is deriving a new class from a base class with extends, so that common code is written once. Polymorphism is calling a virtual method through a base-class handle and getting the behaviour of the actual derived object. Abstraction is defining a common interface in a virtual class with pure virtual methods and leaving the implementation to the derived classes."

The rest of this part covers encapsulation and inheritance. Part 5 covers polymorphism and abstraction.

4.2 Encapsulation

In plain English

Encapsulation has two halves.

1. **Bundling.** The data and the code that works on that data are put in one unit, the class. A packet knows how to print, copy and compare itself.
2. **Information hiding.** The class decides which of its members the outside world may touch. The rest is private. The outside world uses the object only through the methods the class offers.

Technical definition

Encapsulation is the binding of properties and methods into a class together with the restriction of direct access to some of them, using the access qualifiers `local` and `protected` (Section 2.6). The public methods form the class's **interface**; the hidden properties and helper methods form its **implementation**.

Why it is needed

Benefit	Explanation
Data integrity	Hidden data can only change through methods that enforce the rules (for example, a FIFO model that refuses to pop when empty)
Freedom to change the implementation	If users depend only on the public methods, the internal data structure can be replaced without editing any user code
Simpler use	A user needs to learn a handful of public methods, not every internal variable
Easier debugging	When hidden data is wrong, the fault must be in one of the few methods of the class that can write it

Complete example

A reference model of a small FIFO for a scoreboard. The storage is hidden; only `push`, `pop` and the status functions are public.

tb_encapsulation.sv

```
module tb_encapsulation;

  class fifo_model;
    local bit [7:0] mem [$];            // hidden storage
    local int       depth;              // hidden configuration
    local int       overflow_count;     // hidden statistics

    function new(int depth = 16);
      this.depth = depth;
    endfunction

    function bit push(bit [7:0] value);
      if (is_full()) begin
        overflow_count++;
        return 0;                       // rejected: the model stays consistent
      end
      mem.push_back(value);
      return 1;
    endfunction

    function bit pop(output bit [7:0] value);
```

```
      if (is_empty()) return 0;
      value = mem.pop_front();
      return 1;
    endfunction

    function bit is_full();   return mem.size() == depth; endfunction
    function bit is_empty();  return mem.size() == 0;     endfunction
    function int used();      return mem.size();          endfunction
    function int overflows(); return overflow_count;      endfunction
  endclass

  fifo_model fifo;
  bit [7:0]  v;
  bit        ok;

  initial begin
    fifo = new(2);                                  // a 2-entry model

    ok = fifo.push(8'h11);  $display("push 11: ok=%0d used=%0d", ok, fifo.used());
    ok = fifo.push(8'h22);  $display("push 22: ok=%0d used=%0d", ok, fifo.used());
    ok = fifo.push(8'h33);  $display("push 33: ok=%0d used=%0d", ok, fifo.used());

    ok = fifo.pop(v);       $display("pop    : ok=%0d value=%h", ok, v);
    ok = fifo.pop(v);       $display("pop    : ok=%0d value=%h", ok, v);
    ok = fifo.pop(v);       $display("pop    : ok=%0d (empty)", ok);

    $display("overflows=%0d", fifo.overflows());
    // fifo.mem.push_back(8'hFF);                   // ILLEGAL: mem is local
    // fifo.depth = 100;                            // ILLEGAL: depth is local
  end

endmodule
```

EXPECTED OUTPUT

```
push 11: ok=1 used=1
push 22: ok=1 used=2
push 33: ok=0 used=2
pop    : ok=1 value=11
pop    : ok=1 value=22
pop    : ok=0 (empty)
overflows=1
```

Line by line

Line	Explanation
`local bit [7:0] mem [$];`	The queue is an implementation detail. It could be replaced by a fixed array with read and write pointers without changing any code outside the class.
`function bit push(bit [7:0] value);`	The only way to add data. It refuses when the model is full, so `mem` can never exceed `depth`.
`function bit pop(output bit [7:0] value);`	Returns a status and delivers the data through an `output` argument (Module 3).
`is_full()`, `is_empty()`, `used()`, `overflows()`	Read-only "getter" methods. There is no setter for `overflow_count`: nobody outside can falsify the statistics.
Third push	The model holds 2 entries, so the push is rejected and counted.
The two commented lines	Direct access to the internals is rejected by the compiler.

Encapsulation versus abstraction

These two are often confused in interviews.

	Encapsulation	Abstraction
Question it answers	How are the details *protected*?	Which details are *shown* at all?
Focus	Hiding the implementation inside a class	Presenting a simplified interface that many classes share
Mechanism	`class`, `local`, `protected`	`virtual class`, `pure virtual` methods
Level	Inside one class	Across a family of classes
Example	`mem` is `local` in `fifo_model`	Every component has `run()`; callers do not know which component it is

> **Common mistake:** Saying "encapsulation means making everything local". Encapsulation is about a deliberate interface. In transaction classes most fields stay public, because randomization constraints, drivers and scoreboards all need them; what is encapsulated is the *behaviour* (`print`, `copy`, `compare`) together with the data.

Interview questions

Q1. What is encapsulation? Binding data and the methods that operate on it into a single unit (a class), and restricting direct access to the internal data using `local` and `protected`, so that the object is used only through its public methods.

Q2. How is encapsulation achieved in SystemVerilog? By declaring a class, marking implementation details `local` or `protected`, and providing public methods as the interface.

Q3. What is the difference between encapsulation and abstraction? Encapsulation hides the implementation inside a class and protects its data. Abstraction defines a common, simplified interface (typically in an abstract base class) and leaves the implementation to derived classes.

4.3 Inheritance

In plain English

Suppose you have a working class for a bus transaction and now need a "write" transaction that is the same but also carries data. You could copy the class and edit the copy; then every future bug fix has to be made twice. Inheritance lets you say instead: "a write transaction **is a** bus transaction, plus this". The new class automatically has everything the old one has.

Technical definition

Inheritance is the mechanism by which a new class (the **derived**, **child**, **sub** or **extended** class) is defined from an existing class (the **base**, **parent** or **super** class) using the keyword `extends`. The derived class inherits all properties and methods of the base class and may

- **add** new properties and methods,
- **override** (redefine) inherited methods,
- **add or override** constraints (Module 5).

The base class is not modified.

Terminology

Term	Meaning
Base class / parent class / super class	The class being extended
Derived class / child class / subclass / extended class	The class that extends another
"is-a" relationship	A derived object is also a base object: a `write_txn` is a `base_txn`
"has-a" relationship (composition)	A class contains a handle to another class: a `packet` has a `header`. This is not inheritance
Class hierarchy	The tree formed by all the `extends` relationships

Syntax

```
class base_txn;
  ...
endclass

class write_txn extends base_txn;      // single inheritance
  ...
endclass

class burst_txn extends write_txn;     // multilevel inheritance
  ...
endclass
```

What is inherited

Member of the base class	Inherited by the derived class?	Accessible in the derived class's code?
Public properties and methods	Yes	Yes
`protected` properties and methods	Yes	Yes
`local` properties and methods	Yes, they exist in the object	**No**, not by name; only through inherited base-class methods
Static properties and methods	Yes, shared with the base class (one copy for the whole hierarchy)	Yes, unless `local`
Constraints	Yes	Can be overridden by name (Module 5)
The constructor `new`	**Not inherited.** Each class has its own; the derived constructor calls the base constructor (Section 4.4)	

How it works internally

A derived object contains a complete base-class part followed by the members added by each level of derivation.

```
class base_txn;                    bit [7:0] addr;   endclass
class write_txn extends base_txn;  bit [7:0] data;   endclass
class burst_txn extends write_txn; int       len;    endclass

one burst_txn object:
+-------------------------------------------+
|  base_txn part      addr                  |  <- everything declared in base_txn
|-------------------------------------------|
|  write_txn part     data                  |  <- what write_txn added
|-------------------------------------------|
|  burst_txn part     len                   |  <- what burst_txn added
+-------------------------------------------+

class hierarchy:        base_txn
                            ^
                            |   extends
                        write_txn
                            ^
                            |   extends
                        burst_txn
```

Kinds of inheritance

Kind	Meaning	Supported in SystemVerilog?
Single	One class extends one base class	Yes
Multilevel	A extends B, B extends C; any depth	Yes
Hierarchical	Several classes extend the same base class	Yes
Multiple	One class extends two or more base classes	**No.** A class can `extends` only one class. (Since IEEE 1800-2012 a class may additionally `implements` several *interface classes*, which contain only pure virtual method prototypes; see Section 5.5)

Complete example: single and multilevel inheritance

tb_inheritance.sv

```
module tb_inheritance;

  class base_txn;
    bit [7:0] addr;

    function new();
      $display("   base_txn::new");
    endfunction

    function void print();
      $display("addr=%h", addr);
    endfunction
  endclass

  class write_txn extends base_txn;          // single inheritance
    bit [7:0] data;

    function new();
      $display("   write_txn::new");
    endfunction

    function void print_all();
      print();                               // inherited method, used directly
      $display("data=%h", data);
```

```
    endfunction
  endclass

  class burst_txn extends write_txn;          // multilevel inheritance
    int len;

    function new();
      $display("   burst_txn::new");
    endfunction
  endclass

  burst_txn b;

  initial begin
    $display("creating a burst_txn:");
    b = new();

    b.addr = 8'h10;       // declared in base_txn
    b.data = 8'hCD;       // declared in write_txn
    b.len  = 4;           // declared in burst_txn

    b.print_all();        // method from write_txn, which calls print() from base_txn
    $display("len=%0d", b.len);
  end

endmodule
```

EXPECTED OUTPUT

```
creating a burst_txn:
   base_txn::new
   write_txn::new
   burst_txn::new
addr=10
data=cd
len=4
```

Line by line

Line	Explanation
`class write_txn extends base_txn;`	`write_txn` has `addr` and `print()` without declaring them.
`class burst_txn extends write_txn;`	`burst_txn` has everything from both classes above it.
The three `new` functions	None of them calls the base constructor explicitly, yet all three messages appear. The compiler inserts a call to the base-class constructor at the start of each derived constructor (Section 4.4).
Order of the messages	Base first, most-derived last. The object is built from the inside out, so that when a derived constructor runs, the inherited part is already initialised.
`b.addr`, `b.data`, `b.len`	One object, three levels of properties, all accessed the same way.
`print()` inside `print_all()`	An inherited method is called like a method of the class itself.

Inheritance versus composition

	Inheritance ("is-a")	Composition ("has-a")
Syntax	`class b extends a;`	`class b; a a_h; endclass`
Relationship	`b` is a kind of `a`	`b` contains or uses an `a`

	Inheritance ("is-a")	Composition ("has-a")
Access to the other class's members	Direct (public and protected)	Through the handle, public members only
A `b` handle can be assigned to an `a` handle	Yes	No
Examples	`alu_arith_txn extends alu_txn`; `error_test extends base_test`	`alu_env` has an `alu_driver`, an `alu_monitor` and an `alu_scoreboard`

Key point: Use inheritance when the new class really is a specialised version of the old one and should be usable wherever the old one is. Use composition when one object merely uses or owns another. An environment *has* a driver; it is not a driver.

Verification application: Inheritance is how tests are written without touching the environment. A base transaction is randomized by the generator; a test defines `class alu_arith_txn extends alu_txn` with an extra constraint that allows only `ALU_ADD` and `ALU_SUB`, and tells the generator to create that class instead. Likewise `class error_test extends base_test` overrides one method to inject errors. No existing file is edited.

Interview questions

Q1. What is inheritance? Creating a new class from an existing class with `extends`. The derived class gets all properties and methods of the base class and can add new ones or override existing ones.

Q2. Which kinds of inheritance does SystemVerilog support? Single, multilevel and hierarchical. Multiple inheritance of classes is not supported; a class can extend only one class (but may implement several interface classes).

Q3. Are `local` members inherited? They are part of the derived object and are used by the inherited base-class methods, but the derived class's own code cannot refer to them by name.

Q4. Is the constructor inherited? No. Each class has its own constructor. The derived-class constructor calls the base-class constructor, explicitly with `super.new(...)` or implicitly.

Q5. In which order do constructors run when a derived object is created? From the base class down to the most-derived class.

Q6. What is the difference between inheritance and composition? Inheritance models an "is-a" relationship using `extends`. Composition models a "has-a" relationship by holding a handle to another class as a property.

4.4 The super Keyword

In plain English

Inside a derived class, `super` means "the part of me that comes from my parent class". It is used when the derived class has redefined something and still needs the parent's version, and to pass arguments to the parent's constructor.

Technical definition

`super` is a keyword, usable inside a derived class, that refers to the members of the immediate base class. It is used to

1. call the base-class constructor: `super.new(arguments);`
2. call a base-class method that the derived class has overridden: `super.print();`
3. access a base-class property that the derived class has hidden by declaring a property of the same name: `super.kind`.

this versus super

	`this`	`super`
Refers to	The current object, starting the name search in the current class	The same object, but starting the name search in the immediate base class
Typical use	`this.addr = addr;`	`super.new(name);` `super.print();`
Available in	Any non-static method	Non-static methods of a derived class
Can be passed as a handle or returned	Yes	No, `super` is not a handle; it can only be followed by `.member`

Rules for super.new()

Rule	Detail
Position	`super.new(...)` must be the **first executable statement** of the derived constructor
Implicit call	If the derived constructor does not call `super.new`, the compiler inserts `super.new();` (with no arguments) at its start. The same happens if the derived class has no constructor at all
Base constructor with required arguments	The implicit call has no arguments, so it does not compile. The derived class **must** define a constructor that calls `super.new(...)` with the arguments
Alternative form	Arguments for the base constructor may be given in the class header: `class write_txn extends base_txn(8'h10);`
No `super.super`	Only the immediate base class can be named. To reach a grandparent's method, the parent's method must itself call `super.method()`

Order of construction in a derived class

```
d = new();     // class derived extends base

1. memory for the whole object (base part + derived part) is allocated
2. super.new() executes:
     a. base-class property initialisers
     b. base-class constructor body
3. derived-class property initialisers
4. the rest of the derived-class constructor body
```

Step 3 comes after step 2. A derived property initialiser can therefore use values the base constructor has set, and a base constructor cannot rely on derived properties being initialised yet.

Complete example

tb_super.sv

```
module tb_super;

  class base_txn;
    string    name;
    bit [7:0] addr;
    int       x = 1;

    function new(string name, bit [7:0] addr);
      this.name = name;
      this.addr = addr;
      $display("   base_txn::new   x=%0d", x);
      x = 2;
    endfunction

    function void print();
      $display("%s: addr=%h", name, addr);
    endfunction
  endclass

  class write_txn extends base_txn;
    bit [7:0] data;
    int       y = x + 10;                       // initialiser uses an inherited property

    function new(string name, bit [7:0] addr, bit [7:0] data);
      super.new(name, addr);                    // must be first: builds the base part
      this.data = data;
      $display("   write_txn::new  x=%0d y=%0d", x, y);
    endfunction

    function void print();                      // overrides base_txn::print
      super.print();                            // reuse the base-class version
      $display("%s: data=%h", name, data);      // then add what is new
    endfunction
  endclass

  write_txn w;

  initial begin
    w = new("wr0", 8'h40, 8'h7E);
    w.print();
  end

endmodule
```

EXPECTED OUTPUT

```
   base_txn::new   x=1
   write_txn::new  x=2 y=12
wr0: addr=40
wr0: data=7e
```

Line by line

Line	Explanation
`function new(string name, bit [7:0] addr);` in `base_txn`	The base constructor has two required arguments. An implicit `super.new()` would not compile, so the derived class must call it explicitly.
`super.new(name, addr);`	The first statement of the derived constructor. It initialises the inherited part of the object.
First output line	Inside the base constructor, `x` already has its declaration initialiser, 1. The constructor then changes it to 2.

Line	Explanation
`int y = x + 10;`	A derived-class initialiser. It runs **after** `super.new()` has returned, so it sees `x` = 2 and `y` becomes 12.
Second output line	Confirms the order: base initialisers, base body, derived initialisers, derived body.
`function void print();` in `write_txn`	Redefines the inherited method.
`super.print();`	Calls the version in `base_txn`. Without `super.`, the call `print()` would call the method itself and recurse for ever.

Interview trap: "The derived class has no constructor. Is the base constructor still called?" Yes. The compiler provides a default constructor for the derived class, which calls `super.new()`.

Interview trap: "The base class constructor is `function new(int size);` with no default. What happens if I write `class child extends parent; endclass`?" Compile error: the implicit `super.new()` passes no argument. Either give `size` a default value, or write a constructor in `child` that calls `super.new(value)`.

Common mistake: Putting a statement before `super.new(...)`. It is a compile error. Only declarations may precede it.

Common mistake: Forgetting `super.` when extending a method. Writing `print();` inside `write_txn::print` calls `write_txn::print` again: infinite recursion and a stack overflow at run time.

Verification application: Every UVM component constructor is `function new(string name, uvm_component parent); super.new(name, parent); endfunction`, and every overridden phase starts with `super.build_phase(phase);`. Forgetting the `super` call is a standard UVM bug: the base class's work (creating children, applying configuration) silently does not happen.

Interview questions

Q1. What is `super`? A keyword used in a derived class to refer to members of its immediate base class: the base constructor, an overridden method, or a hidden property.

Q2. Why must `super.new()` be the first statement of a constructor? The base-class part of the object must be fully initialised before the derived class's code can safely use inherited members.

Q3. When is an explicit `super.new(...)` call mandatory? When the base-class constructor has arguments without default values.

Q4. Is `super.super.method()` allowed? No. `super` reaches only the immediate base class.

Q5. What is the difference between `this` and `super`? `this` is a handle to the current object and starts name look-up in the current class. `super` refers to the same object but starts look-up in the base class; it is not a handle and cannot be stored or passed.

4.5 Method Overriding

In plain English

A derived class inherits its parent's methods. If one of them does not do quite the right thing for the derived class, the derived class can declare a method with the **same name** and supply its own body. The new version replaces the inherited one for objects of the derived class. This is **overriding**.

Technical definition

Method overriding is the redefinition, in a derived class, of a method that exists in a base class, using the same name. For a `virtual` method the signature (return type, number, types and directions of arguments) must also match. The same can be done with properties: a derived class may declare a property with the same name as a base-class property, which **hides** the base-class property.

Which version runs?

This is the central question, and the answer depends on whether the method is `virtual`.

Method declared in the base class as	Version chosen by	Decided at
Non-virtual (plain `function` / `task`)	The **type of the handle** used for the call	Compile time ("static binding")
`virtual`	The **type of the object** the handle refers to	Run time ("dynamic binding"), Section 5.2

Properties are never virtual: `handle.property` always selects by the type of the handle.

This section demonstrates the non-virtual case, so that the need for `virtual` in Part 5 is clear.

Complete example

tb_override.sv

```
module tb_override;

  class base_txn;
    int kind = 1;

    function void print();
      $display("base_txn::print   kind=%0d", kind);
    endfunction
  endclass

  class write_txn extends base_txn;
    int kind = 2;                              // hides base_txn::kind

    function void print();                     // overrides base_txn::print (non-virtual)
      $display("write_txn::print  kind=%0d  super.kind=%0d", kind, super.kind);
    endfunction
  endclass

  base_txn  b;
  write_txn w;

  initial begin
    w = new();
    w.print();                                 // called through a write_txn handle

    b = w;                                     // base handle, SAME write_txn object
    b.print();                                 // called through a base_txn handle

    $display("b.kind=%0d  w.kind=%0d", b.kind, w.kind);
  end

endmodule
```

EXPECTED OUTPUT

```
write_txn::print  kind=2  super.kind=1
base_txn::print   kind=1
b.kind=1  w.kind=2
```

Line by line

Line	Explanation
`int kind = 2;` in `write_txn`	The object now contains **two** properties called `kind`: the one from `base_txn` (value 1) and the one from `write_txn` (value 2). Inside `write_txn`, the plain name means the derived one; `super.kind` reaches the base one.
`w.print();`	The handle type is `write_txn`, so `write_txn::print` runs.
`b = w;`	A base-class handle may refer to a derived object (upcasting, Section 5.1). No new object is created.
`b.print();`	The handle type is `base_txn` and `print` is not virtual, so the compiler binds the call to `base_txn::print`, **even though the object is a `write_txn`**. Inside that method, `kind` means `base_txn::kind`, which is 1.
`b.kind` and `w.kind`	Property access follows the handle type: 1 through `b`, 2 through `w`. Same object, different answers.

The second output line is usually not what the programmer wanted. The object is a write transaction and should print itself as one, whichever handle is used. Making `print` virtual (Section 5.2) fixes exactly this.

Rules and good practice

Topic	Rule
Overriding a non-virtual method	Allowed with any signature; the derived method simply hides the base method for derived-type handles. Rarely what you want
Overriding a virtual method	The return type and the argument list must match the base-class prototype
Calling the base version	`super.method()`
Access qualifier	An override may be declared with a different qualifier, but keep it the same to avoid surprises
Hiding a property	Legal, but almost always a mistake. Do not reuse a base-class property name in a derived class
Constructors	Not overridden; they are chained with `super.new`
Static methods	Can be redefined in a derived class, but they are not virtual: `class_name::method()` always selects by the named class

Interview trap: "A base handle refers to a derived object and a non-virtual method is called. Which version executes?" The base-class version, because for non-virtual methods the handle type decides. Many candidates answer "the derived one".

Common mistake: Re-declaring a property in the derived class "to change its default value", for example `int kind = 2;`. That creates a second, separate property. To change the default of an inherited property, assign it in the derived constructor: `kind = 2;`.

Interview questions

Q1. What is method overriding? Redefining, in a derived class, a method that already exists in the base class, with the same name (and, for virtual methods, the same signature), so that derived objects use the new implementation.

Q2. How do you call the base-class version of an overridden method? `super.method_name(arguments)` from inside the derived class.

Q3. What happens if a derived class declares a property with the same name as a base-class property? The object has both. The derived one hides the base one inside the derived class; the base one is still reachable with `super.name` or through a base-class handle. Which one `handle.name` selects depends on the type of the handle.

Q4. Without `virtual`, which method is called through a base handle that points to a derived object? The base-class method.

4.6 Method Overloading and Its Limitations

What overloading is

In C++ and Java, **overloading** means having several methods with the **same name in the same class**, distinguished by the number or types of their arguments. The compiler picks the right one for each call:

```
// C++ / Java style -- NOT legal SystemVerilog
void log(string msg);
void log(string msg, int value);
void log(int value);
```

SystemVerilog does not support it

In SystemVerilog a name can be declared only once in a scope. Two methods with the same name in one class are a compile error, whatever their arguments. That includes constructors: a class has exactly one `new`.

```
class logger;
  function void log(string msg);              endfunction
  function void log(string msg, int value);   endfunction   // ERROR: 'log' already declared
endclass
```

Why the language was designed this way: SystemVerilog converts freely and implicitly between integral types of different widths and signedness, between two-state and four-state types, and between integers and reals (Module 1, Section 4.1). Choosing "the best match" among several overloads would be ambiguous for a call such as `f(5)` when both `f(byte)` and `f(logic [31:0])` exist. The language avoids the problem by not allowing it. Default argument values cover most practical needs.

Overloading versus overriding

	Overloading	Overriding
Where	Same class (same scope)	Base class and derived class
Method name	Same	Same
Argument list	Must differ	Must be the same (for virtual methods)
Resolved at	Compile time, from the arguments	Run time for virtual methods, from the object type
Related pillar	Compile-time ("static") polymorphism	Run-time ("dynamic") polymorphism
Supported in SystemVerilog	**No**	**Yes**

Workarounds

Need	SystemVerilog solution	Example
Optional arguments	Default argument values	`function void log(string msg, int value = -1);`
Different argument types	Different method names	`log_int(int v)`, `log_str(string s)`, `log_real(real r)`
Same algorithm for many types	Parameterized class (Module 11)	`class stack #(type T = int);`
Several ways to construct an object	One `new` with defaults, plus static "factory" functions	`static function packet make_read(bit [7:0] addr);`
Different behaviour per class	Overriding with virtual methods	Part 5

Complete example: default arguments in place of overloading

tb_overload_workaround.sv

```
module tb_overload_workaround;

  class logger;
    string prefix;

    // One constructor serves "no argument" and "with argument"
    function new(string prefix = "LOG");
      this.prefix = prefix;
    endfunction

    // One method serves three calling styles
    function void log(string msg, int value = -1, string unit = "");
      if (value == -1) $display("%s: %s", prefix, msg);
      else             $display("%s: %s = %0d%s", prefix, msg, value, unit);
    endfunction

    // A static function used as a second, named way to build an object
    static function logger make_error_logger();
      logger l = new("ERROR");
      return l;
    endfunction
  endclass

  logger l1, l2;

  initial begin
    l1 = new();
    l1.log("start");
    l1.log("count", 5);
    l1.log("delay", 10, "ns");

    l2 = logger::make_error_logger();
    l2.log("mismatches", 3);
  end

endmodule
```

EXPECTED OUTPUT

```
LOG: start
LOG: count = 5
LOG: delay = 10ns
ERROR: mismatches = 3
```

Line by line

Line	Explanation
`function new(string prefix = "LOG");`	Replaces the two constructors `new()` and `new(string)` that C++ would use.
`function void log(string msg, int value = -1, string unit = "");`	One method, three calling styles. The value -1 is used as "no value given", which works here because the logged counts are never negative.
`static function logger make_error_logger();`	A static method that builds and returns a pre-configured object: a "named constructor".
`logger::make_error_logger()`	Called on the class; no object is needed to call it.

Interview trap: "Does SystemVerilog support function overloading?" No. The follow-up is usually "then how do you get the same effect?": default arguments, distinct names, or parameterized classes. A second follow-up: "Is polymorphism still possible?" Yes, run-time polymorphism through virtual-method overriding is fully supported; only compile-time overloading is missing.

Common mistake: Confusing the two words. Over**loading** is same name, different arguments, same class (not available). Over**riding** is same name, same arguments, in a derived class (available).

Interview questions

Q1. What is method overloading? Does SystemVerilog support it? Overloading is defining several methods with the same name but different argument lists in the same scope. SystemVerilog does not support it; each name may be declared once per class.

Q2. What is the difference between overloading and overriding? Overloading: same class, same name, different arguments, resolved at compile time. Overriding: derived class redefines a base-class method with the same name and signature; with `virtual`, resolved at run time.

Q3. Can a class have more than one constructor? No. Use default argument values in the single constructor, or static factory methods.

Q4. How would you write a function that accepts either one or two arguments? Give the second argument a default value.

Part 5: Polymorphism and Abstraction

5.1 Upcasting: Base Handles and Derived Objects

In plain English

Every write transaction is a bus transaction. So wherever a bus transaction is expected, a write transaction can be supplied. In code: a handle whose type is the base class may refer to an object of any class derived from it. Assigning a derived object to a base-class handle is called **upcasting**, because in the usual drawing of a class hierarchy the base class is at the top.

Technical definition

Upcasting is the assignment of a derived-class handle to a base-class handle: `base_h = derived_h;`. It is always legal, needs no cast, and is checked at compile time. The object is not changed, copied or truncated; only the *view* of it is narrower. Through the base-class handle the compiler allows access only to members declared in the base class.

How it works internally

```
write_txn w = new();        base_txn b;        b = w;         // upcast

     w  (type write_txn) ----+
                             +--> +--------------------------+
     b  (type base_txn)  ----+    |  base_txn part   addr    |  <- visible via b and w
                                  |--------------------------|
                                  |  write_txn part  data    |  <- visible through w only
                                  +--------------------------+

b.addr   legal
b.data   COMPILE ERROR: base_txn has no member "data"
         (the data is still in the object; the handle type simply cannot see it)
```

Two types for every handle

To reason about any OOP question, keep two types apart.

	Static type (handle type)	Dynamic type (object type)
What it is	The class the handle variable was declared with	The class of the object the handle currently refers to
Known at	Compile time	Run time
Can change	Never	Every time the handle is assigned
Decides	Which member **names** are legal; which **property** and which **non-virtual** method is selected	Which **virtual** method executes; whether a `$cast` succeeds

Why upcasting is needed

- **Generic containers.** One queue, array or mailbox of `base_txn` handles can hold read, write and burst transactions together.
- **Generic code.** A driver written for `base_txn` can process every present and future derived transaction without modification.
- **Substitution.** A test can replace the transaction a generator creates with a derived one, and the rest of the environment never notices.

All three depend on virtual methods (next section) to be useful: without them, the generic code would always execute base-class behaviour.

Interview questions

Q1. What is upcasting? Assigning a derived-class handle (or object) to a base-class handle. It is implicit and always legal.

Q2. After `base_h = derived_h;`, can `base_h` access a property that exists only in the derived class? No. The compiler uses the handle type to check names, so only base-class members are accessible. The property still exists in the object; to reach it, the handle must be cast back to the derived type with `$cast`.

Q3. Does upcasting create a new object or lose data? No. Both handles refer to the same, complete derived object.

5.2 Virtual Methods and Dynamic Binding

In plain English

Section 4.5 ended with a problem: when a write transaction was printed through a base-class handle, the base-class `print()` ran. The compiler looked at the handle, saw `base_txn`, and wired the call to `base_txn::print` on the spot.

Declaring the method `virtual` changes the rule. The compiler no longer decides. At run time, at the moment of the call, the simulator looks at the **actual object** and runs the version belonging to the object's class.

Technical definition

- A **virtual method** is a function or task declared with the keyword `virtual` in a class. A call to a virtual method is resolved at run time according to the class of the object the handle refers to, not the class of the handle.
- **Binding** is the association of a method call with the code that will execute.
  - **Static (early) binding:** done by the compiler from the handle type. Used for non-virtual methods.
  - **Dynamic (late) binding:** done at run time from the object type. Used for virtual methods.

Syntax

```
class base_txn;
  virtual function void print();       // virtual in the base class
    ...
  endfunction
endclass

class write_txn extends base_txn;
  virtual function void print();       // override; "virtual" optional but recommended
    ...
  endfunction
endclass
```

Rules

Rule	Detail
Once virtual, always virtual	A method declared `virtual` in a class is virtual in every class derived from it. Repeating the keyword in the derived classes is optional; write it anyway for clarity
Signature must match	An override of a virtual method must have the same return type and the same number, types, directions and names of arguments as the base prototype. A mismatch is a compile error
Functions and tasks	Both can be virtual
Not allowed to be virtual	Static methods and the constructor `new`
Properties	Cannot be virtual. Property access always follows the handle type
Calling the base version	`super.method()` is always statically bound to the base class's implementation
Default argument values	Taken from the prototype of the handle's class at compile time. Keep the defaults identical in every override
Access through any handle type	A derived handle calling a virtual method also gets the object's version; "virtual" matters most when the handle type is a base class

How it works internally: the virtual method table

Simulators typically implement dynamic binding with one **virtual method table** ("vtable") per class. The table lists, for each virtual method, the implementation that applies to that class. Every object carries a hidden reference to the table of its own class, set when the object is constructed.

```
class base_txn;
  virtual function void print();  ... endfunction     // virtual
  function void show();           ... endfunction     // NOT virtual
endclass
class write_txn extends base_txn;
  virtual function void print();  ... endfunction     // override
  function void show();           ... endfunction     // hides base show()
endclass

vtable of base_txn                    vtable of write_txn
+---------+--------------------+      +---------+----------------------+
| print   | base_txn::print    |      | print   | write_txn::print     |
+---------+--------------------+      +---------+----------------------+
(show is not virtual: it is in no table)

base_txn b;   write_txn w = new();   b = w;

     b ----> +--------------------+
             | vtable --> write_txn's table
             | addr               |
             | data               |
             +--------------------+

b.print();   1. follow b to the object
             2. follow the object's vtable reference       -> write_txn's table
             3. look up "print"                            -> write_txn::print
             4. call it                                         (dynamic binding)

b.show();    not virtual: the compiler already emitted a direct call to
             base_txn::show, because b is declared as base_txn  (static binding)
```

A virtual call costs one extra table look-up. That cost is negligible, which is why nearly every method in a verification class library is virtual.

Complete example

tb_virtual.sv

```
module tb_virtual;

  class base_txn;
    function void show();                              // non-virtual
      $display("   base_txn::show   (non-virtual)");
    endfunction

    virtual function void print();                     // virtual
      $display("   base_txn::print  (virtual)");
    endfunction
  endclass

  class write_txn extends base_txn;
    function void show();
      $display("   write_txn::show  (non-virtual)");
    endfunction

    virtual function void print();
      $display("   write_txn::print (virtual)");
    endfunction
  endclass

  base_txn  b;
  write_txn w;

  initial begin
    w = new();
    b = w;                        // base handle -> write_txn object

    $display("1. base handle, write_txn object:");
    b.show();
    b.print();

    $display("2. write_txn handle, write_txn object:");
    w.show();
    w.print();

    b = new();                    // base handle -> base_txn object
    $display("3. base handle, base_txn object:");
    b.show();
    b.print();
  end

endmodule
```

EXPECTED OUTPUT

```
1. base handle, write_txn object:
   base_txn::show   (non-virtual)
   write_txn::print (virtual)
2. write_txn handle, write_txn object:
   write_txn::show  (non-virtual)
   write_txn::print (virtual)
3. base handle, base_txn object:
   base_txn::show   (non-virtual)
   base_txn::print  (virtual)
```

Line by line

Call	Handle type	Object type	Virtual?	Decided by	Runs
1. `b.show()`	`base_txn`	`write_txn`	No	Handle	`base_txn::show`
1. `b.print()`	`base_txn`	`write_txn`	Yes	Object	`write_txn::print`
2. `w.show()`	`write_txn`	`write_txn`	No	Handle	`write_txn::show`

Call	Handle type	Object type	Virtual?	Decided by	Runs
2. `w.print()`	`write_txn`	`write_txn`	Yes	Object	`write_txn::print`
3. `b.show()`	`base_txn`	`base_txn`	No	Handle	`base_txn::show`
3. `b.print()`	`base_txn`	`base_txn`	Yes	Object	`base_txn::print`

Block 1 is the important one: the same handle `b`, the same object, two calls, and the two calls go to different classes. The only difference is the keyword `virtual`.

Virtual versus non-virtual method

Property	Non-virtual method	Virtual method
Keyword	None	`virtual`
Binding	Static (compile time)	Dynamic (run time)
Selected by	Type of the handle	Type of the object
Override must match signature	No	Yes
Enables polymorphism	No	Yes
Cost	Direct call	One table look-up
When to use	Small helper that derived classes must never change	Anything a derived class may want to specialise: in practice, almost every method

The three meanings of the keyword virtual

Usage	Meaning	Where covered
`virtual function` / `virtual task`	Method resolved at run time by object type	This section
`virtual class`	Abstract class: cannot be instantiated	Section 5.5
`virtual interface` (`virtual alu_if vif;`)	A handle that refers to an interface instance, so that a class can reach DUT signals	Module 8

They share a keyword and nothing else. "Explain the different uses of `virtual` in SystemVerilog" is a standard interview question.

> **Interview trap:** "The base class declares `virtual function void print();`. The derived class overrides it but leaves out the word `virtual`. Is the derived method virtual?" Yes. Once a method is virtual in a base class it is virtual in all derived classes.

> **Interview trap:** "The base class method is *not* virtual, and the derived class declares the same method *with* `virtual`. What happens through a base handle?" The base version runs. Virtual dispatch starts at the class where the method is first declared virtual; a call through a handle of a class above that point is still statically bound.

> **Common mistake:** Forgetting `virtual` in the base class and then wondering why a derived test's or transaction's override "is never called". When an override seems to be ignored, the first thing to check is whether the base-class method is virtual.

> **Common mistake:** Calling a virtual method from a base-class constructor and expecting derived properties to be ready. In SystemVerilog the derived override *is* called, but the derived class's property initialisers and constructor body have not run yet (Section 4.4), so the override sees default values. Avoid calling virtual methods from constructors.

Interview questions

Q1. What is a virtual method? A method declared with `virtual`. Calls to it are bound at run time to the implementation in the class of the actual object, regardless of the type of the handle used.

Q2. What is the difference between static and dynamic binding? Static binding is done at compile time from the handle's declared type; it applies to non-virtual methods and to properties. Dynamic binding is done at run time from the object's type; it applies to virtual methods.

Q3. Why do we need virtual methods? So that code written in terms of a base class (a driver, a scoreboard, a generator, a container of base handles) automatically executes the specialised behaviour of whatever derived object it is given. Without `virtual`, a base handle would always run base-class code.

Q4. Can a constructor or a static method be virtual? No.

Q5. Does the override of a virtual method need the same arguments? Yes. The return type and argument list must match the base-class declaration.

Q6. What are the uses of the keyword `virtual` in SystemVerilog? Virtual methods (run-time binding), virtual classes (abstract classes), and virtual interfaces (handles to interface instances).

5.3 Polymorphism

In plain English

"Polymorphism" means "many forms". One line of code, `txn.print()`, takes many forms at run time: it prints a read transaction, a write transaction or a burst transaction, depending on what `txn` happens to refer to at that moment. The code that makes the call does not know and does not care which one it is.

Technical definition

Polymorphism is the ability of a single base-class handle to refer to objects of different derived classes, and of a single method call through that handle to execute different implementations depending on the object's actual class. In SystemVerilog it requires three ingredients:

1. **Inheritance:** a base class and one or more derived classes.
2. **Virtual methods:** declared in the base class and overridden in the derived classes.
3. **A base-class handle referring to a derived object** (upcasting).

Remove any one of the three and there is no polymorphism.

Complete example

tb_polymorphism.sv

```
module tb_polymorphism;

  class bus_txn;
    bit [7:0] addr;

    function new(bit [7:0] addr);
      this.addr = addr;
    endfunction

    virtual function void print();
      $display("BUS   addr=%h", addr);
    endfunction

    virtual function int size_in_bytes();
      return 1;
    endfunction
  endclass

  class read_txn extends bus_txn;
    function new(bit [7:0] addr);
      super.new(addr);
    endfunction

    virtual function void print();
      $display("READ  addr=%h", addr);
    endfunction
  endclass

  class write_txn extends bus_txn;
    bit [7:0] data;

    function new(bit [7:0] addr, bit [7:0] data);
      super.new(addr);
      this.data = data;
    endfunction

    virtual function void print();
      $display("WRITE addr=%h data=%h", addr, data);
    endfunction

    virtual function int size_in_bytes();
      return 2;
    endfunction
  endclass

  // Generic code: written once, for the base class only
  function automatic void process(bus_txn t);
    t.print();
    $display("      size=%0d byte(s)", t.size_in_bytes());
  endfunction

  bus_txn   txns [3];          // an array of base-class handles
  read_txn  r;
  write_txn w;

  initial begin
    r = new(8'h10);
    w = new(8'h20, 8'hAB);

    txns[0] = new(8'h00);      // a bus_txn object
    txns[1] = r;               // a read_txn object  (upcast)
    txns[2] = w;               // a write_txn object (upcast)

    foreach (txns[i]) process(txns[i]);
  end

endmodule
```

EXPECTED OUTPUT

```
BUS   addr=00
      size=1 byte(s)
READ  addr=10
      size=1 byte(s)
WRITE addr=20 data=ab
      size=2 byte(s)
```

Line by line

Line	Explanation
`virtual function void print();` in `bus_txn`	Establishes the polymorphic interface.
`class read_txn extends bus_txn;`	Overrides `print` only. It inherits `size_in_bytes` unchanged, so a read reports 1.
`function new(bit [7:0] addr); super.new(addr); endfunction`	Needed because the base constructor has a required argument.
`class write_txn`	Overrides both virtual methods.
`function automatic void process(bus_txn t);`	This function knows only `bus_txn`. It was written without any knowledge of `read_txn` or `write_txn`.
`bus_txn txns [3];`	One array holds three different kinds of object, because each element is a base-class handle.
`foreach (txns[i]) process(txns[i]);`	The same two calls inside `process` produce three different behaviours.

If a `burst_txn extends write_txn` is added next month, `process` and the loop work with it unchanged. That is the practical value of polymorphism: **new behaviour is added by writing new classes, not by editing existing code**.

Without polymorphism

For comparison, the same task without virtual methods needs a type field and a `case` statement in every piece of generic code:

```
case (t.kind)
  KIND_READ : print_read(t);
  KIND_WRITE: print_write(t);
  // every new kind of transaction means editing this statement, everywhere it appears
endcase
```

Polymorphism in a testbench

Where	How polymorphism is used
Generator	Holds an `alu_txn` handle (a "blueprint"). A test assigns a derived-class object to it; the generator's `randomize()` call then applies the derived class's constraints, because `randomize` is virtual (Module 5)
Driver	Receives `alu_txn` handles from the mailbox and calls virtual methods such as `print()`; derived transactions print their extra fields
Tests	`base_test` has virtual tasks; `error_test extends base_test` overrides one. `tb_top` holds a `base_test` handle and calls `run()`

Where	How polymorphism is used
Callbacks	A driver calls `cb.pre_drive(txn)` on a base callback handle; tests register derived callback objects that inject errors or delays
UVM	The factory, phases (`build_phase`, `run_phase`), `do_copy`, `do_compare` and sequences all rely on virtual methods called through base-class handles

> **How to say it in an interview:** "Polymorphism means the same method call behaves differently depending on the object. In SystemVerilog I get it by declaring the method virtual in the base class, overriding it in derived classes, and calling it through a base-class handle that points to a derived object. The method is then chosen at run time by the object type, which is dynamic binding."

Interview questions

Q1. What is polymorphism? The ability of one base-class handle to refer to objects of different derived classes and of one method call to execute the implementation belonging to the actual object.

Q2. What is required for polymorphism in SystemVerilog? Inheritance, a virtual method in the base class that is overridden in the derived class, and a base-class handle pointing to a derived-class object.

Q3. Does SystemVerilog support compile-time polymorphism? Not through method overloading, which it lacks. Parameterized classes give a form of compile-time genericity. Run-time polymorphism through virtual methods is fully supported.

Q4. How does polymorphism help in a verification environment? Components are written once against base classes. Tests change behaviour by supplying derived transactions, tests, callbacks or components, without modifying the environment. This is the basis of testbench reuse and of the UVM factory.

5.4 Downcasting and $cast

In plain English

Upcasting is safe because every write transaction is a bus transaction. The reverse is not safe: a bus-transaction handle might be referring to a write transaction, a read transaction, or a plain bus transaction. Before you may treat it as a write transaction, somebody has to check what the object really is. The compiler cannot check, because the answer is known only at run time. `$cast` performs the check at run time.

Technical definition

Downcasting is the assignment of a base-class handle to a derived-class handle. A direct assignment `derived_h = base_h;` is a **compile error**. It must be done with the dynamic cast `$cast(derived_h, base_h)`, which succeeds only if the object referred to by `base_h` is of the class of `derived_h` or of a class derived from that.

Upcasting versus downcasting

	Upcasting	Downcasting
Direction	Derived handle → base handle	Base handle → derived handle
Syntax	`base_h = derived_h;`	`$cast(derived_h, base_h)`

	Upcasting	Downcasting
Checked at	Compile time	Run time
Can fail	Never	Yes, if the object is not of the destination class (or a subclass of it)
Purpose	Treat different objects uniformly	Regain access to derived-only members

Rules of $cast for class handles

`$cast(dest, src)` succeeds when the **object** `src` refers to could legally be assigned to `dest`:

Object that `src` refers to	Type of `dest`	Result
A `write_txn`	`write_txn`	Succeeds
A `burst_txn` (which extends `write_txn`)	`write_txn`	Succeeds: a burst is a write
A plain `bus_txn`	`write_txn`	**Fails**: the object has no `write_txn` part
A `read_txn` (a sibling class)	`write_txn`	**Fails**
`src` is `null`	Any class type	Succeeds; `dest` becomes `null`

As with enums (Module 1, Section 4.1), `$cast` can be called two ways:

Called as	On success	On failure
Function: `if ($cast(d, b)) ...`	Assigns, returns 1	Returns 0, `d` is left unchanged, no message
Task: `$cast(d, b);`	Assigns	Run-time error, `d` is left unchanged

The static cast `write_txn'(b)` is not used for downcasting class handles; `$cast` is the tool for this job.

Complete example

tb_downcast.sv

```
module tb_downcast;

  class bus_txn;
    bit [7:0] addr;
    virtual function void print();
      $display("BUS   addr=%h", addr);
    endfunction
  endclass

  class read_txn extends bus_txn;
    virtual function void print();
      $display("READ  addr=%h", addr);
    endfunction
  endclass

  class write_txn extends bus_txn;
    bit [7:0] data;
    virtual function void print();
      $display("WRITE addr=%h data=%h", addr, data);
    endfunction
  endclass

  bus_txn   b;
  read_txn  r;
  write_txn w, w2;

  initial begin
    w = new();
```

```
    w.addr = 8'h20;
    w.data = 8'hAB;

    b = w;                              // upcast: always legal
    // w2 = b;                          // COMPILE ERROR: base handle to derived handle
    // $display("%h", b.data);          // COMPILE ERROR: bus_txn has no member data

    if ($cast(w2, b))
      $display("1. cast to write_txn succeeded: data=%h, same object as w: %0d",
               w2.data, w2 == w);

    if (!$cast(r, b))
      $display("2. cast to read_txn failed: the object is a write_txn; r is null: %0d",
               r == null);

    b = new();                          // now b refers to a plain bus_txn
    if (!$cast(w2, b))
      $display("3. cast to write_txn failed: the object is only a bus_txn");
    $display("4. w2 is unchanged: data=%h", w2.data);

    // $cast(w2, b);                    // task form: this line would be a run-time error
  end

endmodule
```

EXPECTED OUTPUT

```
1. cast to write_txn succeeded: data=ab, same object as w: 1
2. cast to read_txn failed: the object is a write_txn; r is null: 1
3. cast to write_txn failed: the object is only a bus_txn
4. w2 is unchanged: data=ab
```

Line by line

Line	Explanation
`b = w;`	Upcast. `b` now refers to the `write_txn` object.
`// w2 = b;`	Rejected by the compiler. From the types alone it cannot know that `b` refers to a `write_txn`.
`// b.data`	Rejected by the compiler: name checking uses the handle type.
`$cast(w2, b)` (line 1)	At run time the object is found to be a `write_txn`. The cast succeeds and `w2` refers to the same object as `w`. Its `data` is reachable again.
`$cast(r, b)` (line 2)	The object is a `write_txn`, which is not a `read_txn`. The function returns 0 and `r` keeps its previous value, `null`.
`b = new();`	`b` is a `bus_txn` handle, so `new()` constructs a `bus_txn` object.
`$cast(w2, b)` (line 3)	The object is only a `bus_txn`; it has no `data`. The cast fails.
Line 4	A failed cast leaves the destination untouched: `w2` still refers to the original write transaction.

What decides success: the object, not the handle

```
bus_txn b;                          bus_txn b;
b = <a write_txn object>            b = <a bus_txn object>

$cast(w2, b)  -> succeeds           $cast(w2, b)  -> fails
the object contains a               the object has no write_txn part;
write_txn part                      letting w2 refer to it would allow
                                    access to a "data" that does not exist
```

Where downcasting is used

Situation	Example
A generic component receives base handles but needs derived-only fields	A scoreboard gets `bus_txn` handles from a mailbox and must read `data` when the item is a write
A virtual `copy()` / `clone()` returns a base-class handle	`$cast(my_copy, original.copy());` (Section 5.6)
Checking the type of an object	`if ($cast(w, t)) ...` works as an "is this a write?" test
UVM	`$cast(req, item)` in drivers and sequences; `$cast(rhs_, rhs)` at the top of every `do_copy` and `do_compare`; the result of `clone()` and of factory `create_object_by_name()`

Interview trap: "Can you assign a base-class handle to a derived-class handle?" Not directly; it is a compile error. With `$cast` it compiles, and it succeeds at run time only if the base handle is actually pointing to an object of the derived class (or one of its subclasses).

Interview trap: "`base_txn b = new(); derived_txn d; $cast(d, b);` Does it work?" It compiles and fails at run time. The object was created as a `base_txn`; no cast can turn it into a `derived_txn`. A cast never changes an object, it only changes the type of the handle through which the object is viewed.

Common mistake: Calling `$cast` as a task in code where failure is a normal possibility (the simulation stops with an error), or calling it as a function and ignoring the result (the destination silently keeps its old value, often `null`, and the next line dies with a null access). Use the function form and handle both outcomes: `if (!$cast(w, t)) $fatal(1, "expected a write_txn");`.

Interview questions

Q1. What is the difference between upcasting and downcasting? Upcasting assigns a derived handle to a base handle; it is implicit and always safe. Downcasting assigns a base handle to a derived handle; it needs `$cast` and is checked at run time.

Q2. Why is `$cast` required for downcasting? Whether the base handle refers to an object of the derived type is known only at run time. `$cast` performs that check; the compiler cannot.

Q3. When does `$cast(derived_h, base_h)` fail? When the object `base_h` refers to is not of the type of `derived_h` or a subclass of it, for example a plain base object or an object of a sibling class.

Q4. What is the difference between `$cast` as a function and as a task? The function returns 1 or 0 and never issues an error. The task issues a run-time error when the cast is illegal. In both cases a failed cast leaves the destination unchanged.

Q5. What happens if the source handle is null? The cast succeeds and the destination becomes `null`.

5.5 Abstract Classes and Pure Virtual Methods

In plain English

Some classes exist only to be extended. "Component" is a useful idea (everything in a testbench that has a name and a `run()` task), but an object that is *only* a component, and not a driver or a monitor or a scoreboard, makes no sense: what would its `run()` do? Such a class is declared **abstract**: you cannot create objects of it. It serves as a contract. It lists the methods every real component must provide, and it can contain the code that all of them share.

A method that the abstract class requires but cannot sensibly implement is declared **pure virtual**: a prototype with no body. Each derived class must write the body.

Technical definition

- A **virtual class** (abstract class) is a class declared with the keyword `virtual` before `class`. It **cannot be instantiated**: calling `new` for it is a compile error. Handles of its type can be declared and can refer to objects of derived classes.
- A **pure virtual method** is a method prototype declared with `pure virtual`, without a body and without `endfunction` / `endtask`. It may be declared only in an abstract class (or in an interface class).
- A non-abstract ("concrete") class derived from an abstract class **must override every pure virtual method** it inherits. If it does not, it is a compile error (or the derived class must itself be declared `virtual`).

Syntax

```
virtual class component;                        // abstract class
  string name;

  function new(string name);                    // may have a constructor
    this.name = name;
  endfunction

  pure virtual task run();                      // no body, no endtask
  pure virtual function string kind();          // no body, no endfunction

  function void report();                       // ordinary method with a body
    $display("%s is a %s", name, kind());
  endfunction
endclass
```

What an abstract class may contain

Item	Allowed in an abstract class?
Properties, including static ones	Yes
A constructor	Yes. It is called by derived constructors through `super.new()`, never directly
Ordinary (non-virtual) methods with bodies	Yes
Virtual methods with bodies (a default implementation)	Yes
Pure virtual methods	Yes, and only here (or in an interface class)

Item	Allowed in an abstract class?
Zero pure virtual methods	Yes. `virtual class` alone is enough to forbid instantiation
Constraints, covergroups, typedefs	Yes

Complete example

tb_abstract.sv

```
module tb_abstract;

  virtual class component;
    string name;

    function new(string name);
      this.name = name;
    endfunction

    pure virtual task run();
    pure virtual function string kind();

    function void report();                       // shared code, written once
      $display("%s is a %s", name, kind());
    endfunction
  endclass

  class demo_driver extends component;
    function new(string name);
      super.new(name);
    endfunction

    virtual task run();
      #10 $display("[%0t] %s: driving", $time, name);
    endtask

    virtual function string kind();
      return "driver";
    endfunction
  endclass

  class demo_monitor extends component;
    function new(string name);
      super.new(name);
    endfunction

    virtual task run();
      #5 $display("[%0t] %s: monitoring", $time, name);
    endtask

    virtual function string kind();
      return "monitor";
    endfunction
  endclass

  component    comps [2];        // handles of the abstract type are allowed
  demo_driver  d;
  demo_monitor m;

  initial begin
    d = new("drv0");
    m = new("mon0");
    comps[0] = d;
    comps[1] = m;

    foreach (comps[i]) comps[i].report();
    foreach (comps[i]) comps[i].run();          // one after the other

    // comps[0] = new("x");                     // ILLEGAL: component is abstract
  end
```

```
endmodule
```

EXPECTED OUTPUT

```
drv0 is a driver
mon0 is a monitor
[10] drv0: driving
[15] mon0: monitoring
```

Line by line

Line	Explanation
`virtual class component;`	Abstract: no object of exactly this class can exist.
`pure virtual task run();`	A requirement, not an implementation. Every concrete component must have a `run` task with this prototype.
`function void report();`	Shared, concrete code in the abstract class. It calls `kind()`, which does not exist here; at run time the call binds to the derived class's `kind()`.
`class demo_driver extends component;`	A concrete class: it implements both pure virtual methods, so it can be instantiated.
`component comps [2];`	Declaring handles of an abstract type is legal and is the whole point: generic code is written against the abstract type.
`comps[i].run();`	The tasks are called in sequence in one thread, so the times add up: the driver finishes at 10, the monitor at 15. (Module 7 shows how to start them in parallel with `fork`.)
The commented line	`new` applied to an abstract handle type is rejected at compile time.

Virtual method versus pure virtual method

Property	Virtual method	Pure virtual method
Declaration	`virtual function void f(); ... endfunction`	`pure virtual function void f();`
Has a body in the declaring class	Yes (a default implementation)	No
Derived class must override	Optional	Mandatory for a concrete class
Allowed in	Any class	Abstract classes and interface classes only
Purpose	Allow specialisation	Force every derived class to provide an implementation

Abstract class versus concrete class

Property	Abstract (`virtual class`)	Concrete class
Can be instantiated with `new`	No	Yes
Can have pure virtual methods	Yes	No
Can declare handles of its type	Yes	Yes
Can be extended	Yes; that is its purpose	Yes
Purpose	Common interface and shared code for a family of classes	Objects that actually exist

Interface classes (awareness level)

IEEE 1800-2012 added the **interface class**: a class-like declaration that contains only pure virtual method prototypes, type declarations and parameters, with no properties and no method bodies. A class can `implements` any number of interface classes in addition to `extends`-ing one ordinary class, which gives the useful part of multiple inheritance without its ambiguities.

```
interface class printable;
  pure virtual function void print();
endclass

class packet implements printable;
  virtual function void print();
    $display("packet");
  endfunction
endclass
```

Do not confuse an `interface class` (an OOP construct) with an `interface` (a bundle of signals, Module 8) or a `virtual interface`. Support for interface classes is good in current versions of the major simulators but was absent in older ones, and the feature is used in UVM 1.2 and later for TLM-style connections; knowing the idea is enough for most interviews.

Interview trap: "Can an abstract class have a constructor?" Yes. It cannot be called directly with `new`, but derived classes call it with `super.new()`.

Interview trap: "Can you declare a handle of an abstract class?" Yes. Only *construction* is forbidden.

Interview trap: "Can an abstract class have non-virtual methods with bodies?" Yes. An abstract class is an ordinary class in every respect except that it cannot be instantiated and may contain pure virtual methods.

Common mistake: Giving a pure virtual method a body or an `endfunction`. A pure virtual declaration is a prototype only and ends at the semicolon.

Verification application: `uvm_component`, `uvm_object`, `uvm_driver` and most other UVM base classes are virtual classes: they are designed to be extended, never instantiated. In a hand-written testbench, a `virtual class base_component` with `pure virtual task run();` guarantees at compile time that every component in the environment can be started the same way.

Interview questions

Q1. What is an abstract class? A class declared with `virtual class`. It cannot be instantiated; it is meant to be extended. It can contain properties, a constructor, concrete methods and pure virtual methods.

Q2. What is a pure virtual method? A method declared with `pure virtual` that has only a prototype and no body. Every concrete derived class must implement it.

Q3. What is the difference between a virtual method and a pure virtual method? A virtual method has an implementation in the base class that derived classes may override. A pure virtual method has no implementation in the base class, can exist only in an abstract (or interface) class, and must be overridden.

Q4. What happens if a derived class does not implement a pure virtual method? It is a compile error, unless the derived class is also declared abstract.

Q5. Can an abstract class have no pure virtual methods? Yes. The `virtual` keyword on the class is sufficient to prevent instantiation.

Q6. What is the difference between a virtual class and a virtual method? A virtual class is an abstract class that cannot be instantiated. A virtual method is a method whose implementation is selected at run time from the object's type.

5.6 Copying in a Class Hierarchy

Section 3.4 promised an explanation of why `copy()` is split in two when classes are extended. The problem and its solution use everything from this part: virtual methods, `super`, upcasting and `$cast`.

The problem

A component holds a base-class handle and wants a copy of whatever object it refers to:

```
base_txn t;          // may refer to a base_txn or to a write_txn
base_txn c;
c = t.copy();        // the copy must be of the SAME class as the object
```

If `copy()` is non-virtual, the base-class version runs, creates a `base_txn`, and the derived fields are lost. So `copy()` must be virtual. But a virtual override must match the signature of the base method, so the derived `copy()` is also declared to return a `base_txn` handle, even though the object it creates is a `write_txn`. (IEEE 1800 does allow an override to declare a derived class as its return type, but tool support for that has been uneven; returning the base type works everywhere and is what UVM does.)

The solution

Method	Virtual	Job
`copy()`	Yes	Creates a new object **of its own class** and returns it as a base-class handle
`copy_fields(rhs)`	Yes	Copies the fields declared in its own class from `rhs`, after calling `super.copy_fields(rhs)` for the inherited ones

tb_virtual_copy.sv

```
module tb_virtual_copy;

  class base_txn;
    bit [7:0] addr;

    virtual function void copy_fields(base_txn rhs);
      this.addr = rhs.addr;
    endfunction

    virtual function base_txn copy();
      base_txn t = new();               // an object of THIS class
      t.copy_fields(this);
      return t;
    endfunction

    virtual function void print(string tag = "");
      $display("[%s] base_txn  addr=%h", tag, addr);
```

```
    endfunction
  endclass

  class write_txn extends base_txn;
    bit [7:0] data;

    virtual function void copy_fields(base_txn rhs);
      write_txn rhs_w;
      super.copy_fields(rhs);                       // inherited fields first
      if ($cast(rhs_w, rhs)) this.data = rhs_w.data; // then the fields added here
    endfunction

    virtual function base_txn copy();
      write_txn t = new();              // an object of THIS class
      t.copy_fields(this);
      return t;                         // upcast to the declared return type
    endfunction

    virtual function void print(string tag = "");
      $display("[%s] write_txn addr=%h data=%h", tag, addr, data);
    endfunction
  endclass

  base_txn  orig, dup;
  write_txn w, w_dup;

  initial begin
    w = new();
    w.addr = 8'h40;
    w.data = 8'h7E;

    orig = w;                           // all the caller has is a base handle
    dup  = orig.copy();                 // virtual: write_txn::copy runs
    dup.print("dup");

    w.data = 8'h00;                     // changing the original...
    dup.print("dup");                   // ...does not change the copy

    if ($cast(w_dup, dup))              // downcast to reach data directly
      $display("w_dup.data=%h  same object as w: %0d", w_dup.data, w_dup == w);
  end

endmodule
```

EXPECTED OUTPUT

```
[dup] write_txn addr=40 data=7e
[dup] write_txn addr=40 data=7e
w_dup.data=7e  same object as w: 0
```

Line by line

Line	Explanation
`virtual function base_txn copy();` in `base_txn`	The polymorphic entry point. Its return type is the base class.
`write_txn t = new();` in `write_txn::copy`	Each class's `copy()` creates an object of its own class. This is the one line a constructor-based copy could never get right, because constructors are not virtual.
`return t;`	A `write_txn` handle returned through a `base_txn` return type: an implicit upcast.
`virtual function void copy_fields(base_txn rhs);`	The argument type must be `base_txn` in every class, because the override must match the base signature.
`super.copy_fields(rhs);`	Each level copies only what it declared and delegates the rest upward. Adding a third level needs no change to the first two.

Line	Explanation
`$cast(rhs_w, rhs)`	Inside `write_txn::copy_fields`, `rhs` is a base handle. A downcast is needed to read `rhs.data`.
`dup = orig.copy();`	`orig` is a `base_txn` handle to a `write_txn` object. Dynamic binding selects `write_txn::copy`, so `dup` refers to a new `write_txn`. The first output line proves it: the derived `print` runs and `data` is present.
`$cast(w_dup, dup)`	The caller downcasts the returned base handle when it needs derived-only members.

This is, in miniature, exactly the `clone()` / `do_copy()` scheme of UVM, including the `$cast(rhs_, rhs)` line that begins every `do_copy`.

Interview questions

Q1. Why must `copy()` be virtual in a class hierarchy? So that calling it through a base-class handle creates an object of the actual derived class and copies the derived fields too. A non-virtual `copy()` would always create and copy only the base part.

Q2. Why does the caller need `$cast` on the result of a virtual `copy()` or `clone()`? The method's return type is the base class (the override must keep the base signature), so the result is a base handle. To assign it to a derived handle a downcast is required.

5.7 Abstraction

In plain English

When you drive a car you use a steering wheel, pedals and a gear lever. Those are the same in a petrol car and an electric car, although what happens underneath is completely different. The controls are an **abstraction**: a small, stable set of operations that hides the mechanism and is the same for every implementation.

Technical definition

Abstraction is the practice of exposing only the essential behaviour of an entity through a well-defined interface while hiding how that behaviour is implemented. In SystemVerilog it is expressed with abstract classes and pure virtual methods: the abstract base class states **what** operations exist; each derived class decides **how** to perform them.

How it shows up in verification

Abstraction	What is exposed	What is hidden
Transaction (Module 6)	"Add 200 and 100" as an object with fields `a`, `b`, `op`	The clock cycles and pin wiggles needed to perform it
Driver	`drive(txn)`	The signal-level protocol
Abstract base component	`run()`	Whether the component drives, monitors or checks

Abstraction	What is exposed	What is hidden
Abstract reference-model operation	`calc(a, b)`	The arithmetic for each opcode (Exercise 4 in Part 6)
Virtual interface (Module 8)	A handle `vif` with named signals	Which physical interface instance, and where in the hierarchy it is
UVM	`build_phase`, `run_phase`, `write()`, `get_next_item()`	Everything the base classes and the scheduler do

Each row lets the code above the abstraction be written once and reused while the code below it changes.

The four pillars in one example

Looking back at `tb_abstract.sv` in Section 5.5:

Pillar	Where it appears
Encapsulation	Each component class bundles its `name` with `run`, `kind` and `report`
Inheritance	`demo_driver` and `demo_monitor` extend `component` and reuse `name`, the constructor logic and `report()`
Polymorphism	`comps[i].run()` and the `kind()` call inside `report()` execute the derived class's code through a `component` handle
Abstraction	`component` is a virtual class with pure virtual `run()` and `kind()`: it defines what a component is without implementing one

Interview questions

Q1. What is abstraction? Showing only the essential interface of something and hiding its implementation. In SystemVerilog it is implemented with abstract (virtual) classes and pure virtual methods.

Q2. How is abstraction different from encapsulation? Encapsulation hides the data and implementation *inside one class* behind its public methods. Abstraction defines a common interface *for a family of classes* and defers the implementation to the derived classes.

Q3. Give an example of abstraction in a testbench. A transaction class abstracts pin-level activity into a single object; a base component class with a pure virtual `run()` task abstracts "something that can be started"; a driver's `drive(txn)` task abstracts the bus protocol.

Part 6: Module 4 Revision and Practice

6.1 Revision Summary

Vocabulary on one page

Term	Meaning
Class	User-defined type containing properties and methods
Object	An instance of a class, created at run time by `new()`
Handle	A variable of class type that refers to an object, or is `null`
Property	A variable declared in a class
Method	A function or task declared in a class (always automatic lifetime)
Constructor	`function new(...)` ; no return type; one per class; runs when an object is created
`this`	Handle to the current object
`super`	Refers to the immediate base class's members
`::`	Scope resolution: static members, class-scoped types, packages, extern bodies
Static property	One copy per class, exists without objects
Static method	Called on the class; no `this` ; only static members; cannot be virtual
`local` / `protected` / public	Own class only / own class and subclasses / everywhere (default)
`const`	Global constant (initialised in the declaration) or instance constant (assigned once in `new`)
`extern`	Prototype in the class, body outside as `class_name::method`
`typedef class x;`	Forward declaration
Shallow copy	`new h` : one level; nested objects shared; constructor not run
Deep copy	User-written `copy()` : nested objects duplicated too
Inheritance	`class d extends b;`
Overriding	Derived class redefines a base method
Overloading	Same name, different arguments, same class: **not supported**
Virtual method	Bound at run time by object type
Upcast	`base_h = derived_h;` always legal
Downcast	`$cast(derived_h, base_h)` ; checked at run time
Abstract class	`virtual class` ; cannot be instantiated
Pure virtual method	Prototype only; must be implemented by concrete derived classes

Rules to remember

Objects and handles

- Declaring a handle creates no object. `new()` creates the object. A handle is `null` until then.
- Using a null handle for a non-static member is a fatal run-time error, never a compile error.
- `h2 = h1` copies the reference. Passing an object to a method, returning it, or putting it in a mailbox or queue also copies only the reference.
- `h1 == h2` compares references, not contents.
- Objects are reclaimed automatically when unreachable. There is no `delete` and no destructor.

Construction order

1. Memory allocated.
2. `super.new()` (base initialisers, then base constructor body).
3. This class's property initialisers.
4. This class's constructor body.

`super.new(...)` must be the first statement; it is inserted automatically with no arguments if you do not write it.

Which method or property is selected?

Member	Selected by
Property	Handle type
Non-virtual method	Handle type (compile time)
Virtual method	Object type (run time)
Static member via `class::`	The named class
`super.method()`	Always the base class's version

Copying

	New outer object	Nested objects	Constructor runs
`p2 = p1;`	No	Shared	No
`p2 = new p1;`	Yes	Shared	No
`p2 = p1.copy();`	Yes	Duplicated	Yes

Casting between class handles

- Derived → base: implicit, always succeeds.
- Base → derived: `$cast` required; succeeds only if the *object* is of the destination class or a subclass.
- `$cast` as a function returns 0/1 silently; as a task it raises a run-time error. A failed cast leaves the destination unchanged.

Abstract classes

- `virtual class`: handles yes, objects no.
- `pure virtual`: no body; only in abstract or interface classes; concrete subclasses must implement all of them.

- An abstract class may have a constructor, properties and ordinary methods.

The keyword `virtual` has three unrelated uses: virtual method, virtual class, virtual interface.

The keyword `static` has two positions: `static function` (static method, legal) and `function static` (static lifetime, illegal in a class).

The four pillars

Pillar	Mechanism	One-line example
Encapsulation	`class`, `local`, `protected`	`fifo_model` hides `mem` behind `push`/`pop`
Inheritance	`extends`, `super`	`write_txn extends base_txn`
Polymorphism	`virtual` methods through base handles	`txns[i].print()`
Abstraction	`virtual class`, `pure virtual`	`component` with `pure virtual task run();`

6.2 Interview Questions and Answers

The questions at the end of each section cover the direct definitions. These are additional.

Basic level

Q1. What are the main features of OOP? Encapsulation, inheritance, polymorphism and abstraction.

Q2. Why is OOP used in verification? Testbenches need objects created at run time (transactions), reuse through inheritance (new tests and transactions derived from existing ones), generic components that work with any derived transaction (polymorphism), and randomization with constraints, which exists only in classes.

Q3. What is the difference between a property and a method? A property is a data member (variable) of a class. A method is a function or task of the class that operates on the properties.

Q4. What is the name of the constructor, and how many can a class have? `new`. Exactly one, because there is no overloading.

Q5. What is `null`? The value of a handle that refers to no object. It is the default value of every handle.

Q6. What keyword is used to derive a class? `extends`.

Q7. What is the default access level of class members? Public.

Q8. Can a function inside a class call a task inside the same class? No. A function cannot call a task, inside or outside a class, because a task may consume time and a function must return immediately (Module 3). A task can call functions and other tasks.

Q9. How do you call a static method? `class_name::method_name(arguments)`. It may also be called through a handle.

Q10. What is a base class and what is a derived class? The base class is the existing class that is extended. The derived class is the new class that inherits from it and adds or overrides members.

Q11. Can you create an object of an abstract class? No. You can only declare handles of it and create objects of concrete derived classes.

Q12. What does `super.new()` do? Calls the constructor of the base class, so that the inherited part of the object is initialised.

Intermediate level

Q13. What is the difference between `new()` and `new h`? `new()` constructs a new object by running the constructor. `new h` (no parentheses) makes a shallow copy of the object that `h` refers to, without running the constructor.

Q14. A class contains a queue of `int` and a handle to another object. After a shallow copy, what is shared? Only the nested object. The queue is a value and is duplicated; the handle is copied, so both objects refer to the same nested object.

Q15. What happens if a derived class does not define a constructor and the base class constructor has a required argument? Compile error. The implicit derived constructor calls `super.new()` with no arguments, which does not match. The derived class must define `new` and call `super.new(arg)`, or the base argument needs a default.

Q16. Can a derived class access a `local` member of its base class? A `protected` one? Not the local one. The protected one, yes.

Q17. Why can a `compare(rhs)` method read `rhs`'s local properties? Access control is per class, not per object. Code inside a class may access local members of any object of that same class.

Q18. If a base method is non-virtual and the derived class overrides it, which runs when called via a derived handle? Via a base handle to the same object? Via the derived handle, the derived method. Via the base handle, the base method.

Q19. Is the keyword `virtual` required in the derived-class override? No. A method that is virtual in the base class stays virtual in all derived classes. Writing the keyword again is recommended for readability.

Q20. Can a static method be overridden? A derived class can declare a static method with the same name, which hides the base one for calls through the derived class name. There is no dynamic dispatch, so it is hiding, not polymorphic overriding.

Q21. Can a virtual method be `local` or `protected`? Yes. The access qualifier and `virtual` are independent. Protected virtual methods are common: "hooks" that only subclasses are meant to override and only the base class calls.

Q22. What is the difference between `this.x` and `super.x`? `this.x` searches for `x` starting in the current class. `super.x` starts the search in the base class, skipping any `x` declared in the current class.

Q23. Why are transaction fields usually public while component internals are local or protected? Transaction fields must be visible to constraints written in tests, to drivers, monitors, scoreboards and coverage. Component internals (queues, counters, state) are implementation details that other code should not depend on.

Q24. What is the difference between a class handle passed as an `input` argument and as a `ref` argument? With `input`, the handle is copied: the method can modify the object, and the caller sees those modifications, but if the method assigns a new object to its argument, the caller's handle is unaffected. With `ref`, the method works on the caller's handle variable itself, so assigning a new object inside the method changes the caller's handle too.

Q25. How do you make sure a class can never be instantiated directly? Declare it as `virtual class`.

Q26. Can an abstract class be derived from a concrete class? Yes. Any class can be extended by a virtual class.

Advanced and scenario-based

Q27. A generator puts 10 transactions into a mailbox. The driver prints 10 transactions with identical values. What is the likely bug? The generator constructs the transaction once, outside its loop, and puts the same handle in the mailbox each time. All ten mailbox entries refer to one object, which holds the values of the last randomization by the time the driver reads it. Fix: `txn = new();` inside the loop, or put `txn.copy()` into the mailbox.

Q28. A scoreboard stores expected transactions in a queue. By the time it compares, the stored values have changed. Why? It stored handles to objects that some other component later modified or reused (often a monitor that keeps one object and overwrites it per transfer). The scoreboard, or the sender, must make a copy.

Q29. A test defines `class my_txn extends alu_txn` and overrides `print()`, but the driver still prints with the base format. Why? Either `print()` is not declared `virtual` in `alu_txn`, so the driver's `alu_txn` handle binds statically to the base method; or the generator is still creating `alu_txn` objects and the derived class is never instantiated.

Q30. The simulation stops with "null object access" inside the driver's `run()` task. List the usual causes. (1) The component object was never constructed (`drv = new(...)` missing in the environment). (2) A handle the component needs was never assigned: the virtual interface or mailbox was not passed in, or was passed before it was created, so `null` was copied. (3) A nested object was not created in the constructor. (4) A `$cast` used as a function failed and its result was ignored, leaving the destination `null`. (5) `try_get` on a mailbox returned nothing and the transaction handle was used anyway.

Q31. Explain what happens, in order, when `d = new();` executes for `class d_c extends b_c`. Memory for the whole object is allocated. `d_c::new` begins and its first action is `super.new()`: the base-class property initialisers are applied and the base constructor body runs. Then the derived-class property initialisers are applied, then the rest of the derived constructor body. Finally the handle is assigned to `d`.

Q32. Why should you avoid calling a virtual method from a constructor? In SystemVerilog the call is dispatched to the derived override, but when a base-class constructor is running the derived class's property initialisers and constructor body have not yet executed. The override therefore runs on a partly initialised object and sees default values.

Q33. How is the "factory" idea related to polymorphism? A factory is a function that creates and returns an object through a base-class handle, where the actual class created can be changed without editing the caller. It works only because the caller uses the object through virtual methods; the caller neither knows nor cares which derived class it received. UVM's `type_id::create()` is this pattern.

Q34. What is a callback and which OOP features does it use? A callback is an object with virtual hook methods (for example `pre_drive(txn)`, `post_drive(txn)`) that a component calls at fixed points. The component holds a base-class callback handle; a test registers a derived object whose overrides inject errors, delays or coverage. It uses inheritance, virtual methods and upcasting, i.e. polymorphism.

Q35. Two objects refer to each other and no other handle refers to either. Are they garbage-collected? Yes. Collection is based on reachability from live handles, and neither object is reachable.

Q36. Can you have a queue or mailbox that holds both `read_txn` and `write_txn` objects? How do you get the derived fields back out? Yes: declare it with the common base type, for example `mailbox #(bus_txn)`.

Derived objects are upcast implicitly on `put`. After `get`, call virtual methods directly, or use `$cast` to a derived handle when a derived-only property is needed.

Q37. What is the difference between `$cast` on enums and `$cast` on class handles? The mechanism is the same (a run-time-checked assignment), but the check differs. For an enum it checks that the integral value is a declared member. For class handles it checks that the source object's class is the destination class or derived from it.

Q38. Why does SystemVerilog not need (and not have) destructors? Memory is reclaimed automatically, which is the main job of a destructor in C++. Other end-of-life actions (reports, checks that queues are empty) are tied to the end of the simulation, not to the death of an object, and are done in explicit methods or in `final` blocks.

Q39. A class has `static int count` and is extended by two derived classes. How many copies of `count` exist? One. A static property declared in a base class is shared by the base class and all classes derived from it. (This does not hold for parameterized classes, where each specialisation has its own statics; Module 11.)

Q40. When would you choose composition over inheritance? When the relationship is "has-a" or "uses-a" and not "is-a". An environment has a driver and a scoreboard; a driver has a virtual interface and a mailbox. Extending a class only to reuse a few of its methods creates a false "is-a" relationship and couples the classes unnecessarily.

6.3 Output-Prediction Questions

Work out each output on paper, then check the answer, then run the code.

P1. Handles

```
module p1;
  class c; int v; endclass
  c a, b;
  initial begin
    a = new();  a.v = 1;
    b = a;      b.v = 2;
    a = new();  a.v = 3;
    $display("%0d %0d", a.v, b.v);
  end
endmodule
```

Answer: `3 2`. `b` refers to the first object, whose `v` was changed to 2 through `b`. `a` was then re-pointed to a second object.

P2. Static

```
module p2;
  class c;
    static int n;
    int id;
    function new();
      id = n;
      n++;
    endfunction
  endclass
  c x, y, z;
  initial begin
    x = new();  y = new();  z = new();
    $display("%0d %0d %0d %0d", x.id, y.id, z.id, c::n);
  end
endmodule
```

Answer: `0 1 2 3`. Each object records the counter before incrementing it. The single static `n` ends at 3.

P3. Shallow copy

```
module p3;
  class inner; int v = 5; endclass
  class outer;
    int   k = 1;
    inner sub = new();
  endclass
  outer a, b;
  initial begin
    a = new();
    b = new a;
    b.k     = 2;
    b.sub.v = 9;
    $display("%0d %0d %0d %0d", a.k, a.sub.v, b.k, b.sub.v);
  end
endmodule
```

Answer: `1 9 2 9`. `k` is duplicated by the shallow copy; the `inner` object is shared, so the change through `b.sub` is seen through `a.sub`.

P4. Constructor chain

```
module p4;
  class A; function new(); $display("A"); endfunction endclass
  class B extends A; function new(); $display("B"); endfunction endclass
  class C extends B; endclass
  C c;
  initial c = new();
endmodule
```

Answer: `A` then `B`. `C` has no constructor, so the implicit one calls `B::new`, which implicitly calls `A::new` first.

P5. Virtual call from a non-virtual method

```
module p5;
  class A;
    virtual function void f(); $display("A.f"); endfunction
    function void g(); $display("A.g"); f(); endfunction
  endclass
  class B extends A;
    function void f(); $display("B.f"); endfunction
    function void g(); $display("B.g"); endfunction
  endclass
  A a;
  B b;
  initial begin
    b = new();
    a = b;
    a.g();
  end
endmodule
```

Answer: `A.g` then `B.f`. `g` is non-virtual, so the handle type `A` selects `A::g`. Inside it, `f()` means `this.f()`, and `f` is virtual (still virtual in `B` without the keyword), so the object type `B` selects `B::f`.

P6. Property hiding

```
module p6;
  class A;
    int x = 1;
    function int get(); return x; endfunction
  endclass
  class B extends A;
    int x = 2;
  endclass
  A a;
  B b;
  initial begin
    b = new();
    a = b;
    $display("%0d %0d %0d", a.x, b.x, b.get());
  end
endmodule
```

Answer: `1 2 1`. Properties follow the handle type. `get()` is inherited from `A` and was compiled inside `A`, where `x` means `A::x`.

P7. $cast

```
module p7;
  class A; endclass
  class B extends A; endclass
  class C extends A; endclass
  A a;
  B b;
  C c;
  initial begin
    b = new();
    a = b;
    $display("%0d %0d", $cast(c, a), $cast(b, a));
    a = new();
    $display("%0d %0d", $cast(b, a), b == null);
  end
endmodule
```

Answer: `0 1` then `0 0`. The object is a `B`: casting to the sibling `C` fails, casting to `B` succeeds. Then `a` refers to a plain `A` object: the cast to `B` fails, and a failed cast leaves `b` unchanged, so it is not null.

P8. Shadowing

```
module p8;
  class P;
    int v;
    function void set(int v);  v = v;       endfunction
    function void set2(int v); this.v = v;  endfunction
  endclass
  P p;
  initial begin
    p = new();
    p.set(5);   $display("%0d", p.v);
    p.set2(7);  $display("%0d", p.v);
  end
endmodule
```

Answer: `0` then `7`. In `set`, both names are the argument, so the property is untouched.

P9. super chain

```
module p9;
  class A;
    virtual function void f(); $display("A"); endfunction
  endclass
  class B extends A;
    virtual function void f(); super.f(); $display("B"); endfunction
  endclass
  class C extends B;
    virtual function void f(); $display("C"); super.f(); endfunction
  endclass
  A a;
  C c;
  initial begin
    c = new();
    a = c;
    a.f();
  end
endmodule
```

Answer: `C`, `A`, `B`. Dynamic binding selects `C::f`, which prints `C` and calls `B::f`; that calls `A::f` first (prints `A`) and then prints `B`.

P10. Initialiser order

```
module p10;
  class A;
    int x = 10;
    function new(); x = x + 1; endfunction
  endclass
  class B extends A;
    int y = x * 2;
    function new(); y = y + 1; endfunction
  endclass
  B b;
  initial begin
    b = new();
    $display("%0d %0d", b.x, b.y);
  end
endmodule
```

Answer: `11 23`. Base initialiser (`x` = 10), base constructor (`x` = 11), derived initialiser (`y` = 22), derived constructor (`y` = 23).

6.4 Find-the-Error and Debugging Questions

Compile-time errors

E1.

```
class base_txn; endclass
class write_txn extends base_txn; endclass
base_txn  b = new();
write_txn w;
initial w = b;
```

A base handle cannot be assigned to a derived handle. Use `$cast(w, b)` (which will then fail at run time here, because the object is a `base_txn`).

E2.

```
class account;
  local int balance;
endclass
account a = new();
initial a.balance = 100;
```

`balance` is local and cannot be accessed outside the class. Add a public method, or remove `local`.

E3.

```
class counter;
  int count;
  static function void incr();
    count++;
  endfunction
endclass
```

A static method cannot access a non-static property. Make `count` static, or make `incr` non-static.

E4.

```
virtual class shape;
  pure virtual function int area();
endclass
shape s = new();
```

An abstract class cannot be instantiated. Create an object of a concrete derived class.

E5.

```
virtual class shape;
  pure virtual function int area();
endclass
class square extends shape;
  int side;
endclass
square sq = new();
```

`square` is concrete but does not implement the pure virtual method `area()`. Add `virtual function int area(); return side * side; endfunction`.

E6.

```
class base_txn;
  function new(int size);
  endfunction
endclass
class write_txn extends base_txn;
  function new();
  endfunction
endclass
```

The implicit `super.new()` passes no argument to a constructor that requires one. Write `super.new(8);` (or any suitable value) as the first statement, or give `size` a default.

E7.

```
class logger;
  function void log(string s);        endfunction
  function void log(string s, int v); endfunction
endclass
```

Overloading is not supported; `log` is declared twice. Use one method with a default argument, or two names.

E8.

```
class write_txn extends base_txn;
  function new(string name);
    $display("creating %s", name);
    super.new(name);
  endfunction
endclass
```

`super.new` must be the first statement in the constructor. Move the `$display` after it.

E9.

```
class packet;
  const int id;
  function new(int id);  this.id = id;  endfunction
  function void renumber(int id);  this.id = id;  endfunction
endclass
```

An instance constant may be assigned only in the constructor. `renumber` is illegal; remove it or drop `const`.

E10.

```
class base_txn;
  virtual function void print(string tag);
  endfunction
endclass
class write_txn extends base_txn;
  virtual function int print();
    return 0;
  endfunction
endclass
```

The override of a virtual method must have the same return type and argument list as the base declaration.

E11.

```
class id_gen;
  static int next_id;
  static function int get();
    return this.next_id;
  endfunction
endclass
```

`this` does not exist in a static method. Write `return next_id;`.

E12.

```
class packet;
  function void new();
  endfunction
endclass
```

The constructor has no return type. Write `function new();`.

Run-time bugs (the code compiles)

D1.

```
alu_txn txn;
initial begin
  txn.a = 8'h10;
  txn.print();
end
```

`txn` was never constructed. Null object access at run time. Add `txn = new();`.

D2.

```
alu_txn txn = new();
repeat (5) begin
  txn.a = $urandom_range(0, 255);
  gen2drv_mbx.put(txn);
end
```

One object, five references in the mailbox. Construct a new object in every iteration.

D3.

```
class packet;
  header hdr;
  bit [7:0] data;
endclass
packet p = new();
initial p.hdr.id = 8'h01;
```

`hdr` is a handle property and is `null`: creating the packet does not create its header. Add `hdr = new();` in a constructor or as a declaration initialiser.

D4.

```
expected = mon_txn;            // scoreboard keeps the handle it was given
...
if (actual == expected) $display("PASS");
```

Two problems. The scoreboard stores a handle to an object the monitor may reuse; it should store a copy. And `==` compares handles, not contents; use `actual.compare(expected)`.

D5.

```
write_txn w;
void'($cast(w, base_h));
$display("%h", w.data);
```

The result of `$cast` is discarded. If `base_h` does not refer to a `write_txn`, `w` stays `null` and the next line is a null access. Test the return value.

D6.

```
class base_test;
  task run();  $display("base test");  endtask
endclass
class error_test extends base_test;
  task run();  $display("error test");  endtask
endclass
base_test t;
error_test e = new();
initial begin  t = e;  t.run();  end
```

Prints "base test". `run` is not virtual, so the base handle selects the base task. Declare it `virtual task run();` in `base_test`.

D7.

```
function packet copy();
  packet p = new();
  p.data = this.data;
  p.hdr  = this.hdr;
  return p;
endfunction
```

This is a shallow copy written by hand: both packets share one header. Use `p.hdr = this.hdr.copy();`.

D8.

```
class write_txn extends base_txn;
  virtual function void print();
    print();
    $display("data=%h", data);
  endfunction
endclass
```

`print()` calls itself for ever. The intention was `super.print();`.

6.5 Coding Exercises with Solutions

Attempt each exercise before reading the solution. The solutions are in `examples/module-04/`.

Exercise 1: an encapsulated stack

Write a class `int_stack` with hidden storage, a maximum depth given to the constructor, `push` (returns 0 when full), `pop` (returns 0 when empty) and `depth`.

ex01_stack.sv

```
module ex01_stack;

  class int_stack;
    local int items [$];
    local int max_depth;

    function new(int max_depth = 4);
      this.max_depth = max_depth;
    endfunction

    function bit push(int value);
      if (items.size() >= max_depth) return 0;
      items.push_back(value);
      return 1;
    endfunction

    function bit pop(output int value);
      if (items.size() == 0) return 0;
      value = items.pop_back();
      return 1;
    endfunction

    function int depth();
      return items.size();
    endfunction
  endclass

  int_stack s;
  int       v;
  bit       ok;

  initial begin
    s = new(3);
    ok = s.push(10);
    ok = s.push(20);
    ok = s.push(30);
    ok = s.push(40);   $display("push 40 accepted? %0d", ok);
    ok = s.pop(v);     $display("pop -> %0d", v);
    ok = s.pop(v);     $display("pop -> %0d", v);
    $display("depth = %0d", s.depth());
  end

endmodule
```

EXPECTED OUTPUT

```
push 40 accepted? 0
pop -> 30
pop -> 20
depth = 1
```

A stack is last-in first-out, so `pop_back` returns the most recent value. The fourth push is refused because the depth limit is 3. Outside code cannot touch `items` at all.

Exercise 2: a linked list built from handles

Write a `node` class holding an `int` and a handle to the next node, and an `int_list` class with `append`, `sum` and `print`.

ex02_linked_list.sv

```
module ex02_linked_list;

  class node;
    int  value;
    node next;                          // a handle of the class's own type

    function new(int value);
      this.value = value;
    endfunction
  endclass

  class int_list;
    local node head;
    local node tail;
    local int  count;

    function void append(int value);
      node n = new(value);
      if (head == null) head = n;       // first node
      else              tail.next = n; // link after the current last node
      tail = n;
      count++;
    endfunction

    function int sum();
      int  total = 0;
      node n     = head;
      while (n != null) begin
        total += n.value;
        n = n.next;
      end
      return total;
    endfunction

    function void print();
      string s = "";
      node   n = head;
      while (n != null) begin
        s = {s, $sformatf("%0d ", n.value)};
        n = n.next;
      end
      $display("list: %s(count=%0d)", s, count);
    endfunction
  endclass

  int_list lst;

  initial begin
    lst = new();
    lst.append(5);
    lst.append(7);
    lst.append(9);
    lst.print();
    $display("sum = %0d", lst.sum());
  end

endmodule
```

EXPECTED OUTPUT

```
list: 5 7 9 (count=3)
sum = 21
```

Each `node` object stays alive because the previous node (or `head`) refers to it. Walking the list is a loop that follows `next` until it reaches `null`. If `head` were set to `null`, every node would become unreachable and be reclaimed.

Exercise 3: shallow versus deep copy with an array and a nested object

A `frame` has a `header` object and a dynamic payload. Write `copy()` and `compare()`, then show the difference between a deep copy and `new f1`.

ex03_frame_copy.sv

```
module ex03_frame_copy;

  class header;
    bit [7:0] id;
    function header copy();
      header h = new();
      h.id = this.id;
      return h;
    endfunction
  endclass

  class frame;
    header    hdr;
    bit [7:0] payload [];

    function new(int size = 0);
      hdr     = new();
      payload = new[size];
    endfunction

    function frame copy();
      frame f = new();
      f.hdr     = this.hdr.copy();     // nested object: must be copied explicitly
      f.payload = this.payload;        // array: assignment copies all elements
      return f;
    endfunction

    function bit compare(frame rhs);
      if (rhs == null) return 0;
      return (hdr.id == rhs.hdr.id) && (payload == rhs.payload);
    endfunction
  endclass

  frame f1, f2, f3;

  initial begin
    f1 = new(3);
    f1.hdr.id  = 8'd7;
    f1.payload = '{8'd1, 8'd2, 8'd3};

    f2 = f1.copy();                                    // deep copy
    $display("after deep copy: equal=%0d", f1.compare(f2));

    f2.hdr.id     = 8'd8;
    f2.payload[0] = 8'd9;
    $display("f1: id=%0d payload[0]=%0d   f2: id=%0d payload[0]=%0d   equal=%0d",
             f1.hdr.id, f1.payload[0], f2.hdr.id, f2.payload[0], f1.compare(f2));

    f3 = new f1;                                       // shallow copy
    f3.payload[1] = 8'd50;                             // array was duplicated
    f3.hdr.id     = 8'd99;                             // header is shared
    $display("after shallow copy: f1.payload[1]=%0d f1.hdr.id=%0d",
             f1.payload[1], f1.hdr.id);
  end

endmodule
```

EXPECTED OUTPUT

```
after deep copy: equal=1
f1: id=7 payload[0]=1   f2: id=8 payload[0]=9   equal=0
after shallow copy: f1.payload[1]=2 f1.hdr.id=99
```

The last line is the instructive one. After the shallow copy, changing `f3`'s payload did not touch `f1` (arrays are values), but changing `f3`'s header changed `f1`'s header (handles are references).

Exercise 4: a polymorphic reference model

Model three ALU operations as classes derived from an abstract `alu_operation` with a pure virtual `calc`. Evaluate all of them through an array of base handles.

ex04_alu_operations.sv

```
module ex04_alu_operations;

  virtual class alu_operation;
    pure virtual function bit [8:0] calc(bit [7:0] a, bit [7:0] b);
    pure virtual function string op_name();
  endclass

  class add_op extends alu_operation;
    virtual function bit [8:0] calc(bit [7:0] a, bit [7:0] b);
      return {1'b0, a} + {1'b0, b};
    endfunction
    virtual function string op_name();
      return "ADD";
    endfunction
  endclass

  class sub_op extends alu_operation;
    virtual function bit [8:0] calc(bit [7:0] a, bit [7:0] b);
      return {1'b0, a} - {1'b0, b};
    endfunction
    virtual function string op_name();
      return "SUB";
    endfunction
  endclass

  class xor_op extends alu_operation;
    virtual function bit [8:0] calc(bit [7:0] a, bit [7:0] b);
      return {1'b0, a ^ b};
    endfunction
    virtual function string op_name();
      return "XOR";
    endfunction
  endclass

  alu_operation ops [3];
  add_op        op_a;
  sub_op        op_s;
  xor_op        op_x;

  initial begin
    op_a = new();  op_s = new();  op_x = new();
    ops[0] = op_a;
    ops[1] = op_s;
    ops[2] = op_x;

    foreach (ops[i])
      $display("%s(200, 100) = %0d", ops[i].op_name(), ops[i].calc(8'd200, 8'd100));
    $display("%s(5, 10) = %0d", ops[1].op_name(), ops[1].calc(8'd5, 8'd10));
  end

endmodule
```

EXPECTED OUTPUT

```
ADD(200, 100) = 300
SUB(200, 100) = 100
XOR(200, 100) = 172
SUB(5, 10) = 507
```

200 is `1100_1000` and 100 is `0110_0100`; their XOR is `1010_1100` = 172. 5 - 10 in nine bits is `1_1111_1011` = 507, with bit 8 acting as the borrow, exactly as the DUT in Module 1 produces. Adding a new operation means adding one class; the loop does not change.

Exercise 5: a test hierarchy

Write a `base_test` whose `run()` task calls two virtual "hook" functions, and an `error_test` that overrides the hooks. Run both through one base handle.

ex05_test_hierarchy.sv

```
module ex05_test_hierarchy;

  class base_test;
    string name = "base_test";

    virtual function int num_txns();
      return 2;
    endfunction

    virtual function string stimulus(int i);
      return $sformatf("normal txn %0d", i);
    endfunction

    virtual task run();
      $display("--- %s ---", name);
      for (int i = 0; i < num_txns(); i++) begin
        #10;
        $display("[%0t] %s", $time, stimulus(i));
      end
    endtask
  endclass

  class error_test extends base_test;
    function new();
      name = "error_test";              // change the inherited property's value
    endfunction

    virtual function int num_txns();
      return 3;
    endfunction

    virtual function string stimulus(int i);
      if (i == 1) return "ERROR injection";
      return super.stimulus(i);
    endfunction
  endclass

  base_test  t;
  base_test  b;
  error_test e;

  initial begin
    b = new();
    e = new();
    t = b;  t.run();
    t = e;  t.run();
  end

endmodule
```

EXPECTED OUTPUT

```
--- base_test ---
[10] normal txn 0
[20] normal txn 1
--- error_test ---
[30] normal txn 0
[40] ERROR injection
[50] normal txn 2
```

`error_test` does not override `run()` at all. The inherited `run()` calls `num_txns()` and `stimulus()` virtually, so the derived versions are used. The base class fixes the overall flow and the derived class changes only the details: this arrangement is called the *template method* pattern and is how UVM phases and `do_*` hooks work.

Exercise 6: a singleton configuration object

Write a class `tb_config` of which only one object can ever exist. All components obtain it through a static method.

ex06_singleton.sv

```
module ex06_singleton;

  class tb_config;
    local static tb_config m_inst;      // the one and only object
    int num_txns = 10;
    bit verbose;

    local function new();               // nobody outside can call new
    endfunction

    static function tb_config get();
      if (m_inst == null) m_inst = new();
      return m_inst;
    endfunction
  endclass

  tb_config c1, c2;

  initial begin
    c1 = tb_config::get();              // first call creates the object
    c1.num_txns = 25;

    c2 = tb_config::get();              // later calls return the same object
    $display("c2.num_txns=%0d  same object=%0d", c2.num_txns, c1 == c2);

    // c1 = new();                      // ILLEGAL: the constructor is local
  end

endmodule
```

EXPECTED OUTPUT

```
c2.num_txns=25  same object=1
```

The static method is the only route to the object, and the static handle keeps the object alive for the whole simulation. This combines static properties, static methods, access control and handle semantics. UVM's `uvm_root` and its factory are singletons of this kind.

Further practice (no solutions given)

1. Extend `alu_txn` from Section 3.6 with `class alu_arith_txn extends alu_txn` that adds a `bit expect_carry` field. Make `print`, `copy` and `compare` virtual in the base class and override them using the `copy_fields` scheme of Section 5.6.
2. Write a class `timer` with a static property counting the objects created and a static function returning the count. Create five objects in a loop and print the count without using any handle.
3. Write an abstract class `shape` with `pure virtual function real area();` and concrete classes `rectangle` and `circle`. Put objects of both in a queue of `shape` handles and print the total area.
4. Take `tb_override.sv` from Section 4.5, add `virtual` to the base `print`, and predict the new output before running it.
5. Write a function `function automatic int count_writes(bus_txn q[$]);` that uses `$cast` to count how many elements of a queue of base handles are `write_txn` objects.
6. Create a class with a constructor that prints a message. Make one object, one shallow copy and one deep copy (through a `copy()` method). Predict how many times the message is printed.
7. Write two classes that hold handles to each other, using a forward `typedef class`. Then set the only external handle to `null` and explain what happens to both objects.

6.6 Concepts You Must Understand Before Moving Forward

Do not start Module 5 until you can answer each of these without the notes.

Classes, objects, handles

- [ ] I can explain class, object and handle, and draw the memory picture of `p2 = p1;`.
- [ ] I know what `null` is and what happens on a null access.
- [ ] I can list what `new()` does, in order, for a base class and for a derived class.
- [ ] I can explain why `this.addr = addr;` needs `this`.
- [ ] I can explain why a new object must be constructed inside a generator loop.

Class members

- [ ] I can explain static properties and methods and the restriction on static methods.
- [ ] I can distinguish `static function` from `function static`.
- [ ] I know the access rules for `local`, `protected` and public members.
- [ ] I can write an `extern` method and a forward `typedef class`.
- [ ] I can explain how objects are freed.

Copying

- [ ] I can state the difference between handle assignment, shallow copy and deep copy, with a diagram.
- [ ] I can write `copy()` and `compare()` for a class with a nested object.
- [ ] I know why `==` on handles is not a content comparison.

Inheritance and polymorphism

- [ ] I can name and define the four pillars with a SystemVerilog example of each.
- [ ] I can explain `super.new()` and when it must be written explicitly.

- [ ] I can explain overriding versus overloading and why overloading is missing.
- [ ] Given a handle type, an object type and whether a method is virtual, I can say which method runs.
- [ ] I can explain upcasting, downcasting and when `$cast` fails.
- [ ] I can explain abstract classes and pure virtual methods, and the three uses of `virtual`.

6.7 What Comes Next

Module 5 adds randomization to classes: `rand` and `randc` properties, constraint blocks, `randomize()`, `pre_randomize()` and `post_randomize()`, and how constraints are inherited and overridden in derived classes. It uses the `alu_txn` class from Section 3.6 and depends directly on inheritance and virtual methods from this module, because `randomize()` is itself a virtual method.